

A Major Project Report on

Smart ERP Billing System

A Desktop Application for Automated Billing, Inventory and Business Management



Submitted in partial fulfillment of the requirements for the award of
the degree of

Bachelor of Technology
in
Computer Science and Engineering

Submitted By:

1. Amit Kumar(20)
2. Yerrabati Boya Ashok(52)
3. Bhipender
4. Rahul Goswami
5. Shweta

Under the Guidance of:

Dhawal Deshpande
Computer Science And Engineering, Department

Department of Computer Science and Engineering
University / Lovely Professional University
Phagwara, Punjab, India

Academic Year: 2023 – 2027

Acknowledgement

The successful completion of this project would not have been possible without the guidance, support, and encouragement of several people, and I take this opportunity to express my sincere gratitude to all of them.

I would like to express my deepest gratitude to my project guide, [**Guide Name**], for the continuous support, patience, motivation, and immense knowledge shared throughout the duration of this project. Their guidance helped me at every stage of research, design, development, and documentation of this work.

I am also thankful to [**Head of Department Name**], Head of the Department of Computer Science and Engineering, for providing the necessary infrastructure and a healthy academic environment that made this project possible.

I extend my sincere thanks to all the faculty members of the department for their valuable suggestions and constant encouragement during the course of this project. I would also like to thank the technical and non-teaching staff for their support during the development and testing phases.

I am grateful to my family and friends for their unwavering support, patience, and encouragement throughout this journey. Finally, I would like to thank everyone, directly or indirectly, who contributed to the successful completion of this project.

[**Student Name**]

Abstract

Small and medium sized businesses such as grocery stores, medical shops, retail outlets, clothing stores, electronics shops, hardware shops and wholesalers still rely heavily on manual or semi-manual methods for managing billing, inventory, and customer records. This dependency on paper registers, spreadsheets, or fragmented tools leads to inefficiency, human error, poor scalability, and a general lack of real time visibility into business operations.

The **Smart ERP Billing System** is a desktop-based Enterprise Resource Planning (ERP) application developed in Python, designed to automate and streamline the core operational processes of small and medium businesses. The system integrates billing, inventory management, customer relationship management, invoice generation, sales reporting, and administrative settings into a single unified platform.

The application is built using a modern graphical interface powered by **CustomTkinter** and **Tkinter**, offering a clean, responsive and user-friendly experience. Data persistence is handled through an embedded **SQLite** relational database, which ensures reliable, ACID-compliant local storage without the overhead of a client-server database engine. The system follows a **Three-Layer Architecture**, separating the presentation layer, business logic layer, and data access layer, thereby improving maintainability, testability and scalability of the codebase.

Key features of the system include secure user authentication, a real-time interactive dashboard, automated inventory tracking with low-stock alerts, a searchable customer database, a fast and accurate billing and invoice generation module, tax and discount computation, and detailed sales and inventory reports. The system has been developed using Visual Studio Code as the primary IDE and targets the Windows operating system, while remaining largely portable to other platforms due to the cross-platform nature of Python and SQLite.

This report documents the complete lifecycle of the project, including requirement analysis, feasibility study, system design, database design, implementation details, testing methodology, and results. Extensive testing including unit, integration, system, performance and security testing was carried out to validate the correctness, robustness, and reliability of the application. The results demonstrate that the Smart ERP Billing System significantly reduces billing time, minimizes human error, and

provides business owners with actionable insights through automated reporting, making it a practical and scalable solution for small and medium enterprises transitioning from manual record keeping to digital operations.

Keywords: Enterprise Resource Planning, Billing System, Inventory Management, Python, CustomTkinter, SQLite, Desktop Application, Three-Layer Architecture, Invoice Generation, Business Automation, Customer Relationship Management, Sales Reporting.

Contents

Acknowledgement	i
Abstract	ii
List of Figures	x
List of Tables	xi
Abbreviations	xii
1 Introduction	1
1.1 Overview	1
1.2 What is ERP?	2
1.3 Billing Systems	2
1.4 Inventory Management	3
1.5 Business Automation	3
1.6 Need for an ERP-Based Billing Solution	3
1.7 Importance of the Project	4
1.8 Background and Current Industry Practice	5
1.9 Problem Overview	5
2 Problem Statement, Existing and Proposed System	6
2.1 Problem Statement	6
2.1.1 Current Manual System	6
2.1.2 Problems in Existing Billing Practices	6

2.1.3	Inventory Issues	7
2.1.4	Paper Records and Human Errors	7
2.1.5	Stock Management Challenges	7
2.1.6	Customer Management Challenges	8
2.1.7	Invoice Generation Challenges	8
2.1.8	Business Losses	8
2.1.9	Time Consumption	8
2.1.10	Accuracy, Scalability and Security Issues	8
2.2	Existing System	9
2.2.1	Overview of Existing Approaches	9
2.2.2	Advantages of Existing Systems	9
2.2.3	Disadvantages of Existing Systems	9
2.2.4	Limitations of Existing Systems	10
2.3	Proposed System	10
2.3.1	Goals	10
2.3.2	Objectives	10
2.3.3	Architecture	10
2.3.4	Working	11
2.3.5	Advantages of the Proposed System	11
2.3.6	Features	11
2.3.7	Benefits	12
2.3.8	Expected Outcomes	12
2.4	Project Objectives	12
2.5	Scope of the Project	13
2.6	Applications	14
2.7	Software Requirements	14
2.8	Hardware Requirements	15
2.9	Feasibility Study	15
2.9.1	Economic Feasibility	15
2.9.2	Technical Feasibility	15
2.9.3	Operational Feasibility	15
2.9.4	Legal Feasibility	16
2.9.5	Schedule Feasibility	16
3	Literature Survey and Requirement Analysis	17
3.1	Literature Survey	17
3.1.1	Summary of Literature Survey Findings	19
3.2	Requirement Analysis	19
3.2.1	Functional Requirements	19

3.2.2	Non-Functional Requirements	20
3.2.3	User Requirements	21
3.2.4	Business Requirements	21
3.2.5	System Requirements	21
4	Software Development Life Cycle and System Design	22
4.1	Software Development Life Cycle Models	22
4.1.1	Waterfall Model	22
4.1.2	Agile Model	22
4.1.3	Spiral Model	22
4.1.4	Prototype Model	23
4.1.5	Selected Model for This Project	23
4.2	System Architecture	23
4.3	Module Diagram	24
4.4	Workflow Diagram	25
4.5	High-Level and Low-Level Design	25
4.5.1	High-Level Design	25
4.5.2	Low-Level Design	26
4.6	Database Architecture	26
4.7	Component Diagram	27
4.8	Deployment Diagram	27
5	System Modeling: DFD and UML Diagrams	28
5.1	Data Flow Diagrams	28
5.1.1	Context Diagram (Level 0)	28
5.1.2	Level 1 DFD	29
5.1.3	Level 2 DFD: Billing Process	30
5.2	Use Case Diagram	30
5.3	Sequence Diagram: Billing Process	31
5.4	Activity Diagram: Billing Process	32
5.5	Class Diagram	33
5.6	State Diagram: Invoice Lifecycle	33
5.7	Entity Relationship Diagram	34
6	Database Design	35
6.1	Overview	35
6.2	Users Table	35
6.3	Customers Table	36
6.4	Products Table	36
6.5	Invoices Table	37

6.6	InvoiceItems Table	37
6.7	Suppliers, Purchases, Sales, Reports and Settings Tables	38
6.8	Primary Keys, Foreign Keys and Relationships	39
6.9	Normalization	39
6.10	Sample Table Creation SQL	39
7	GUI Design and Module Explanation	41
7.1	GUI Design Philosophy	41
7.2	Login Screen	41
7.3	Dashboard	42
7.4	Inventory Screen	42
7.5	Customer Screen	43
7.6	Billing Screen	43
7.7	Invoice Screen	44
7.8	Reports Screen	44
7.9	Settings Screen	45
7.10	Profile and Logout	45
7.11	Complete Module Explanation	45
7.11.1	Authentication Module	45
7.11.2	Inventory Module	46
7.11.3	Customer Module	46
7.11.4	Billing Module	46
7.11.5	Invoice Module	46
7.11.6	Sales Module	46
7.11.7	Reports Module	46
7.11.8	Settings Module	47
7.11.9	Database Module	47
8	Algorithms and Mathematical Model	48
8.1	Algorithms Used	48
8.1.1	Product Search Algorithm	48
8.1.2	Sorting Algorithm	48
8.1.3	SQLite Query Optimization	48
8.1.4	Invoice Calculation Algorithm	49
8.1.5	Tax Calculation	49
8.1.6	Discount Calculation	49
8.1.7	Stock Update Algorithm	49
8.2	Mathematical Model	50
8.2.1	Input	50

8.2.2	Process	50
8.2.3	Output	50
8.2.4	Constraints	50
9	Implementation	51
9.1	Implementation Overview	51
9.2	Project Folder Structure	51
9.3	Database Connection Module	52
9.4	Authentication Module Implementation	53
9.5	Inventory Module Implementation	54
9.6	Billing Module Implementation	55
9.7	GUI Implementation Example	56
9.8	Event Handling	57
9.9	Invoice Generation	57
10	Testing	58
10.1	Testing Overview	58
10.2	Black Box Testing	58
10.3	White Box Testing	58
10.4	Unit Testing	58
10.5	Integration Testing	59
10.6	System Testing	59
10.7	Acceptance Testing	59
10.8	Performance Testing	59
10.9	Security Testing	59
10.10	Test Cases	60
11	Screenshots	63
12	Advantages, Limitations and Future Enhancements	66
12.1	Advantages	66
12.2	Limitations	67
12.3	Future Enhancements	68
13	Project Timeline, Risk, Maintenance and Security	70
13.1	Project Timeline	70
13.2	Risk Analysis	71
13.3	Maintenance	71
13.4	Security Features	72
13.5	Performance Analysis	72

14 Results and Discussion	73
14.1 Results	73
14.2 Discussion	73
15 Conclusion	75
16 References	76
References	76
Appendix A Database Schema	79
Appendix B Folder Structure	80
Appendix C Installation Instructions	81
Appendix D Commands	82
Appendix E Requirements File	83
Appendix F License	84
Appendix G GitHub and README	85
Glossary	86

List of Figures

4.1	Three-Layer Architecture of the Smart ERP Billing System	24
4.2	Module Diagram of the Smart ERP Billing System	24
4.3	High-Level Workflow of a Billing Transaction	25
4.4	Component Diagram	27
4.5	Deployment Diagram (Single Machine Deployment)	27
5.1	Context Level (Level 0) Data Flow Diagram	28
5.2	Level 1 Data Flow Diagram	29
5.3	Level 2 Data Flow Diagram (Billing Sub-Process)	30
5.4	Use Case Diagram	31
5.5	Sequence Diagram for Billing Process	31
5.6	Activity Diagram for Billing Process	32
5.7	Class Diagram	33
5.8	State Diagram for Invoice Lifecycle	33
5.9	Entity Relationship Diagram	34
7.1	Login Screen	41
7.2	Dashboard Screen	42
7.3	Inventory Management Screen	42
7.4	Customer Management Screen	43
7.5	Billing Screen	43
7.6	Invoice Preview Screen	44
7.7	Reports Screen	44
7.8	Settings Screen	45
7.9	Profile Screen	45
11.1	Login Screen Screenshot	63
11.2	Dashboard Screenshot	63
11.3	Inventory Screenshot	63
11.4	Customer Management Screenshot	64
11.5	Billing Screenshot	64
11.6	Invoice Screenshot	64
11.7	Reports Screenshot	64

11.8 Settings Screenshot	65
13.1 Gantt Chart Representing Project Timeline (Weeks)	71

List of Tables

2.1	Software Requirements	14
2.2	Hardware Requirements	15
3.1	Literature Survey Summary	19
6.1	Users Table Structure	35
6.2	Customers Table Structure	36
6.3	Products Table Structure	36
6.4	Invoices Table Structure	37
6.5	InvoiceItems Table Structure	37
6.6	Suppliers Table Structure	38
6.7	Purchases Table Structure	38
6.8	Sales Summary Table Structure	38
6.9	Settings Table Structure	38
10.1	Test Cases	62
13.1	Project Timeline Summary	70
13.2	Risk Analysis	71

Abbreviations

Abbreviation	Full Form
ERP	Enterprise Resource Planning
GUI	Graphical User Interface
DB	Database
SQL	Structured Query Language
SQLite	Structured Query Language Lite (embedded database engine)
CRUD	Create, Read, Update, Delete
IDE	Integrated Development Environment
OS	Operating System
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
UML	Unified Modeling Language
API	Application Programming Interface
UI	User Interface
UX	User Experience
SDLC	Software Development Life Cycle
GST	Goods and Services Tax
QR	Quick Response (Code)
MVC	Model View Controller
ACID	Atomicity, Consistency, Isolation, Durability
PK	Primary Key
FK	Foreign Key

Chapter 1

Introduction

1.1 Overview

Every business, irrespective of its size, depends on efficient management of three core operational pillars: sales and billing, inventory, and customer relationships. In the current competitive business environment, the speed and accuracy with which a business can process a sale, update its stock, and generate a valid invoice directly affects customer satisfaction, profitability, and long-term sustainability. Traditionally, small and medium businesses such as grocery stores, medical shops, and retail outlets have relied on manual registers, handwritten bills, or basic spreadsheet tools to manage these operations. While such methods may have been adequate for very small scale operations, they quickly become a bottleneck as the business grows, leading to lost sales records, incorrect stock counts, delayed billing, and an overall lack of visibility into business performance.

Enterprise Resource Planning (ERP) systems were originally developed for large manufacturing organizations to integrate disparate business functions such as finance, procurement, manufacturing, and human resources into a single unified system. Over time, the underlying principle of ERP – the integration of business processes and data into one coherent platform – has proven equally valuable for small and medium enterprises (SMEs). However, most commercial ERP solutions available in the market are either too expensive, too complex, or too heavyweight for a small shop owner who simply wants to generate bills quickly, keep track of stock, and understand which products are selling well.

The **Smart ERP Billing System** presented in this report is a lightweight, desktop-based ERP application specifically designed to bridge this gap. It brings together billing, inventory management, customer management, invoice generation, and reporting into a single, easy-to-use desktop application built with Python. Unlike cloud-based or subscription-based ERP software, this system runs entirely on a local machine, uses an embedded SQLite database, and requires no internet connectivity

or recurring subscription fees, making it especially suitable for small business owners who want a low-cost, dependable, and private billing solution.

1.2 What is ERP?

Enterprise Resource Planning refers to a category of software systems that help organizations manage and integrate the important parts of their business operations. An ERP system typically integrates planning, purchasing, inventory, sales, marketing, finance, and human resources into a unified system of record. The core idea behind ERP is simple: rather than having separate, disconnected systems for each department, all data flows through a single, centralized database, ensuring consistency, reducing duplication, and providing management with an accurate, real-time view of the entire business.

For small businesses, a full-scale ERP implementation is often unnecessary and impractical. What such businesses typically need is a focused subset of ERP functionality – specifically billing, inventory, and customer management – delivered through a simple, affordable, and reliable tool. The Smart ERP Billing System is designed around this principle: it takes the core philosophy of ERP (integration, centralization, and automation of business data) and applies it in a scaled-down, practical form suited to the needs of small and medium businesses.

1.3 Billing Systems

A billing system is the component of a business application responsible for calculating the cost of goods or services purchased by a customer, applying applicable taxes and discounts, and generating a formal invoice or receipt. In a traditional manual environment, billing is performed by hand: a cashier or shop owner writes down each item purchased, looks up its price, calculates the total, and hands the customer a handwritten or loosely printed bill. This process is slow, error-prone, and leaves little to no digital record for later analysis.

An automated billing system, by contrast, retrieves product prices directly from a centralized product database, automatically calculates subtotals, taxes, and discounts, and instantly generates a professional, itemized invoice that can be printed or stored digitally. Automated billing not only speeds up the checkout process but also creates a permanent, searchable digital record of every transaction, which is essential for accounting, auditing, and business analysis.

1.4 Inventory Management

Inventory management refers to the process of ordering, storing, tracking, and controlling a business's stock of goods. Effective inventory management ensures that a business has enough stock to meet customer demand without overstocking, which ties up capital and increases the risk of spoilage or obsolescence, particularly for perishable goods sold by grocery and medical stores.

Manual inventory management, typically performed using paper ledgers or basic spreadsheets, suffers from several critical weaknesses: stock counts are updated infrequently and inaccurately, there is no automatic alert when stock runs low, and reconciling physical stock with recorded stock is a tedious, error-prone, and time-consuming process. The Smart ERP Billing System automates this process by decrementing stock levels automatically whenever a sale is billed, and by incrementing stock levels whenever new stock is purchased or received, ensuring that the recorded inventory always reflects the true state of the business in near real time.

1.5 Business Automation

Business automation refers to the use of technology to perform recurring business tasks or processes with minimal human intervention, replacing manual, repetitive work with software-driven workflows. Automation reduces the likelihood of human error, increases processing speed, and frees up business owners and employees to focus on higher value activities such as customer service and business growth.

In the context of this project, business automation manifests in several ways: automatic calculation of invoice totals, taxes, and discounts; automatic updating of stock levels after every transaction; automatic generation of daily, weekly, and monthly sales reports; and automatic flagging of products that have fallen below a defined reorder threshold. Each of these automated processes eliminates a task that would otherwise have to be performed manually, often multiple times per day, by the business owner or staff.

1.6 Need for an ERP-Based Billing Solution

The need for a system such as the Smart ERP Billing System arises directly from the limitations of manual and semi-manual business management practices that are still widespread among small and medium enterprises. Several factors drive this need:

- **Growing transaction volumes:** As a business grows, the number of daily transactions increases, and manual billing simply cannot keep pace without introducing delays and errors.
- **Customer expectations:** Modern customers expect quick, accurate billing and a professional printed or digital invoice, even from small shops.
- **Competitive pressure:** Businesses that adopt digital tools gain an edge over competitors who continue to rely on manual processes, both in terms of efficiency and customer trust.
- **Need for accurate records:** Tax compliance, accounting, and business analysis all depend on accurate, complete, and easily retrievable transaction records, which manual systems cannot reliably provide.
- **Inventory visibility:** Businesses need to know, at any given moment, exactly what stock they have, what needs to be reordered, and what is not selling, information that is extremely difficult to maintain accurately by hand.
- **Cost constraints:** Many commercial ERP and billing solutions are priced for larger enterprises and are simply out of reach for a small shop owner, creating a market gap for an affordable, locally-hosted alternative.

1.7 Importance of the Project

This project is important because it directly addresses a widespread and persistent operational problem faced by a very large segment of the economy: small and medium sized retail and service businesses. Unlike large enterprises that can afford dedicated IT departments and expensive ERP licenses, small businesses often operate with extremely tight margins and limited technical expertise. A solution that is free of recurring license costs, easy to install, easy to use, and does not require an internet connection or specialized IT support has significant practical value.

Furthermore, this project serves as a comprehensive demonstration of how modern software engineering principles – layered architecture, relational database design, modular programming, and rigorous testing – can be applied to build a real-world, production-quality desktop application using entirely open-source and freely available technologies, namely Python, CustomTkinter, Tkinter, and SQLite.

1.8 Background and Current Industry Practice

Across a wide range of industries – from small neighborhood grocery stores to medical shops, hardware dealers, wholesalers, and clothing boutiques – there remains a substantial reliance on manual and semi-digital processes for day-to-day operations. Many shop owners still use printed bill books, handwritten stock registers, and separate notebooks or spreadsheet files to track customer credit. Even where computers are present, they are frequently used only for basic tasks such as typing invoices in a word processor, without any integration between billing, inventory, and customer data.

This fragmented approach means that critical business information exists in silos: sales data in one place, stock counts in another, and customer records in yet another, with no automatic synchronization between them. As a direct consequence, business owners often lack a clear, real-time picture of their overall business performance, making it difficult to make timely, informed decisions about purchasing, pricing, and staffing.

1.9 Problem Overview

At a high level, the core problem this project addresses can be summarized as follows: *small and medium businesses lack access to an affordable, easy-to-use, integrated software solution that automates billing, inventory management, and customer record keeping, resulting in inefficiency, errors, and lost business insight.* The following chapters explore this problem in greater depth, examine existing systems and their shortcomings, and present the design, implementation, and evaluation of the Smart ERP Billing System as a practical solution to this widespread problem.

Chapter 2

Problem Statement, Existing and Proposed System

2.1 Problem Statement

Small and medium businesses that continue to rely on manual or semi-manual methods for billing and inventory management face a wide range of operational challenges that directly affect their efficiency, profitability, and ability to scale. This section presents a detailed examination of the specific problems that motivate the development of the Smart ERP Billing System.

2.1.1 Current Manual System

In a typical manual billing environment, a cashier or shop owner records each sale in a physical bill book or notebook, manually calculating the price of each item, the total quantity, and the final amount payable, often with the help of a calculator. Stock levels are tracked separately, usually in a different register, and are updated only periodically – sometimes only once a day or even once a week – rather than in real time as sales occur. Customer information, where it is tracked at all, is often recorded on loose paper or in a separate notebook, with no easy way to search, filter, or analyze this data.

2.1.2 Problems in Existing Billing Practices

- **Slow transaction processing:** Manually looking up prices, calculating totals, and writing out a bill for each customer takes considerably longer than an automated system, leading to long queues and customer dissatisfaction, particularly during peak hours.
 - **Human errors:** Manual calculation of totals, taxes, and discounts is highly prone
-

to arithmetic mistakes, which can result in undercharging (a direct loss to the business) or overcharging (which damages customer trust).

- **Illegible or lost records:** Handwritten bills are sometimes illegible, and paper records can be easily lost, damaged, or destroyed, resulting in permanent loss of transaction history.
- **No centralized record:** Because each bill exists only as a physical piece of paper, there is no easy way to search past transactions, analyze sales trends, or reconcile accounts at the end of the day.

2.1.3 Inventory Issues

Manual inventory tracking suffers from delayed updates, inaccurate counts, and the complete absence of automated reorder alerts. Because stock levels are not updated in real time as sales occur, a shop owner has no reliable way of knowing, at any given moment, exactly how much stock of a particular product remains. This often results in either stockouts (leading to lost sales and dissatisfied customers) or overstocking (which ties up working capital unnecessarily, and for perishable goods, can lead to spoilage and direct financial loss).

2.1.4 Paper Records and Human Errors

Paper-based record keeping is inherently fragile and inefficient. Physical documents can be lost, damaged by water or fire, misfiled, or simply become disorganized over time as their volume grows. Furthermore, extracting any kind of aggregate information from paper records – for example, total sales for a particular month, or the best-selling product of the quarter – requires manually reviewing and tallying potentially hundreds or thousands of individual paper bills, a task that is both time-consuming and highly susceptible to human error.

2.1.5 Stock Management Challenges

Without an automated system, reconciling recorded stock with actual physical stock (a process known as stock-taking or stock audit) becomes a major undertaking that can require shutting down business operations for hours or even days. Discrepancies between recorded and actual stock, whether due to theft, spoilage, damage, or simple counting errors, often go undetected for extended periods, compounding the financial impact.

2.1.6 Customer Management Challenges

Manual systems make it very difficult to maintain a useful customer database. Information such as customer contact details, purchase history, and outstanding credit balances is either not recorded at all or recorded inconsistently across different notebooks or loose papers, making it nearly impossible to identify repeat customers, track outstanding dues, or run any kind of targeted promotional activity.

2.1.7 Invoice Generation Challenges

Generating a clear, professional, itemized invoice by hand is time-consuming and inconsistent in quality. Handwritten invoices often lack important details such as a unique invoice number, a clear breakdown of taxes, or the business's registration details, all of which may be required for accounting or regulatory compliance purposes.

2.1.8 Business Losses

Collectively, the problems outlined above – slow service, calculation errors, lost or illegible records, inventory mismanagement, and poor customer tracking – translate directly into financial losses for the business, whether through lost sales, incorrect pricing, wasted stock, or the inability to make informed purchasing and pricing decisions.

2.1.9 Time Consumption

Every manual process described above consumes significant staff time that could otherwise be spent serving customers or growing the business. Time spent recalculating totals, searching for past bills, manually updating stock registers, or reconciling accounts at the end of the day represents a direct and recurring operational cost.

2.1.10 Accuracy, Scalability and Security Issues

Manual systems are inherently limited in accuracy due to human error, and they do not scale well as transaction volume grows – the same manual processes that are merely inconvenient for a small shop become completely unworkable for a rapidly growing business. Furthermore, paper records offer essentially no security: they can be read, altered, or removed by anyone with physical access, with no audit trail, no access control, and no way to detect or prevent unauthorized changes.

2.2 Existing System

2.2.1 Overview of Existing Approaches

Currently, small businesses generally rely on one of the following approaches for managing billing and inventory: (1) fully manual, paper-based systems; (2) basic spreadsheet-based systems using tools such as Microsoft Excel; or (3) generic, often expensive, commercial point-of-sale (POS) or ERP software that is not specifically tailored to the needs or budget of very small businesses.

2.2.2 Advantages of Existing Systems

- Manual systems require no technical training or computer literacy to operate.
- Manual systems have no upfront software or hardware cost beyond basic stationery.
- Spreadsheet-based systems offer some improvement over pure paper records, allowing for basic calculations and sorting.
- Commercial ERP or POS software, where affordable and adopted, often provides a comprehensive feature set covering accounting, payroll, and multi-branch operations.

2.2.3 Disadvantages of Existing Systems

- Manual systems are slow, error-prone, and offer no automated reporting or analytics.
- Spreadsheet-based systems are not designed for concurrent, transactional use and quickly become unwieldy as data volume grows; they also lack proper relational data integrity, referential constraints, and built-in invoice generation.
- Commercial ERP and POS solutions are frequently expensive, require internet connectivity or ongoing subscription fees, and are often over-engineered with features that a small business does not need, resulting in a steep learning curve.
- None of the existing low-cost approaches provide a tightly integrated combination of billing, inventory, customer management, and reporting specifically tailored to a small business's actual workflow.

2.2.4 Limitations of Existing Systems

The fundamental limitation shared by all of the existing approaches is the absence of a truly integrated, affordable, and easy-to-use system that a non-technical small business owner can install and start using immediately, without ongoing costs, without an internet dependency, and without a steep learning curve. This gap in the market is precisely what the Smart ERP Billing System is designed to fill.

2.3 Proposed System

2.3.1 Goals

The primary goal of the proposed system is to deliver a lightweight, integrated, desktop-based ERP billing solution that automates the core operational workflows of a small or medium business – specifically billing, inventory management, customer management, and reporting – while remaining affordable, easy to install, and easy to use, even for users with limited computer literacy.

2.3.2 Objectives

The proposed system aims to eliminate manual calculation errors, provide real-time inventory visibility, maintain a searchable and persistent customer database, generate professional invoices automatically, and provide business owners with clear, automatically generated sales and inventory reports that support informed decision making.

2.3.3 Architecture

The proposed system follows a **Three-Layer Architecture**, consisting of a Presentation Layer (built with CustomTkinter and Tkinter), a Business Logic Layer (implemented in Python, responsible for validation, calculations, and workflow orchestration), and a Data Access Layer (responsible for all interaction with the underlying SQLite relational database). This separation of concerns is discussed in detail in Chapter 4.

2.3.4 Working

When a cashier or business owner opens the application, they are first authenticated through a secure login screen. Once logged in, they are presented with an interactive dashboard summarizing key business metrics. From here, they can navigate to the inventory module to view or update stock, the customer module to manage customer records, or the billing module to create a new sale. During billing, products are selected from the inventory (searchable by name or code), quantities are entered, and the system automatically calculates the subtotal, applicable taxes, and any discounts, before generating a final invoice that can be printed or saved. All of these actions automatically update the underlying SQLite database, ensuring that inventory levels, sales records, and customer histories remain perpetually accurate and up to date.

2.3.5 Advantages of the Proposed System

- Eliminates manual calculation errors through automated billing logic.
- Maintains real-time, always-accurate inventory records.
- Provides a centralized, searchable customer database.
- Generates professional, consistent invoices automatically.
- Produces automated sales and inventory reports for better business insight.
- Runs entirely offline, with no recurring subscription costs.
- Uses a lightweight embedded database (SQLite), requiring no separate database server installation.
- Provides a modern, intuitive graphical interface that is easy to learn.

2.3.6 Features

Secure login and authentication; interactive dashboard with key business metrics; product and inventory management with low-stock alerts; customer relationship management; fast and accurate billing with barcode-style product search; automatic tax and discount calculation; invoice generation and printing; sales and inventory reporting; and a settings module for business configuration.

2.3.7 Benefits

The proposed system reduces the time required to process each sale, minimizes financial losses due to calculation or stock errors, improves customer service through faster checkout, and gives business owners clear visibility into their operations through automatically generated reports, all without the cost and complexity typically associated with commercial ERP solutions.

2.3.8 Expected Outcomes

It is expected that businesses adopting the Smart ERP Billing System will experience a measurable reduction in billing time per transaction, a significant reduction in stock discrepancies, improved accuracy in financial records, and an overall improvement in operational efficiency and customer satisfaction.

2.4 Project Objectives

The specific objectives of this project are as follows:

1. To design and develop a secure, role-based login and authentication system.
2. To design an interactive, informative dashboard summarizing key business metrics.
3. To develop a complete inventory management module supporting addition, editing, deletion, and searching of products.
4. To implement automated low-stock alerts based on configurable reorder thresholds.
5. To develop a customer management module for maintaining customer profiles and purchase history.
6. To develop a fast, accurate, and user-friendly billing module.
7. To implement automatic calculation of subtotal, tax, and discount for every invoice.
8. To design and implement an invoice generation module capable of producing printable invoices.
9. To integrate a relational SQLite database for reliable, persistent local data storage.
10. To design a normalized relational database schema covering all core entities of the system.

11. To implement a Three-Layer Architecture separating presentation, business logic, and data access.
12. To develop a sales reporting module capable of generating daily, weekly, and monthly reports.
13. To develop an inventory reporting module highlighting stock levels and reorder recommendations.
14. To implement a settings module allowing configuration of business details, tax rates, and preferences.
15. To ensure data integrity through proper use of primary keys, foreign keys, and referential constraints.
16. To provide a modern, responsive graphical user interface using CustomTkinter and Tkinter.
17. To ensure the application performs efficiently even with a large volume of historical transaction data.
18. To implement basic security measures, including password hashing and input validation.
19. To conduct thorough testing, including unit, integration, system, and performance testing.
20. To document the complete software development lifecycle, from requirement analysis to deployment.
21. To ensure the system is easy to install, configure, and use by non-technical business owners.
22. To design the system to be extensible, allowing future addition of features such as GST compliance, barcode scanning, and cloud synchronization.

2.5 Scope of the Project

The scope of the Smart ERP Billing System encompasses the design, development, testing, and documentation of a desktop application covering user authentication, inventory management, customer management, billing and invoice generation, sales and inventory reporting, and system settings. The system is intended for single-location small and medium businesses operating on the Windows operating system, using a

local SQLite database for data storage. The scope does not include multi-branch synchronization, cloud hosting, mobile applications, or integration with external payment gateways in the current version, though these are identified as potential future enhancements in Chapter 12.

2.6 Applications

The Smart ERP Billing System is applicable to a wide range of small and medium business types, including but not limited to grocery stores, medical and pharmacy shops, general retail stores, clothing and apparel boutiques, electronics shops, hardware and tool shops, wholesale distributors, and other small businesses that require billing, inventory tracking, and basic customer management functionality.

2.7 Software Requirements

Table 2.1: Software Requirements

Component	Specification
Operating System	Windows 10 / Windows 11
Programming Language	Python 3.10 or higher
GUI Framework	CustomTkinter, Tkinter
Database	SQLite 3
IDE	Visual Studio Code
Version Control	Git and GitHub
Additional Libraries	Pillow (image handling), reportlab / fpdf (PDF invoice generation), tkcalendar (date selection)

2.8 Hardware Requirements

Table 2.2: Hardware Requirements

Component	Minimum Specification
Processor	Dual-core 1.6 GHz or higher
RAM	4 GB minimum (8 GB recommended)
Storage	500 MB free disk space (excluding data growth)
Display	1366 x 768 resolution or higher
Printer (optional)	Any standard inkjet, laser, or thermal receipt printer

2.9 Feasibility Study

2.9.1 Economic Feasibility

The proposed system is built entirely using free and open-source technologies, including Python, CustomTkinter, Tkinter, and SQLite, all of which are available at no licensing cost. This eliminates recurring software licensing fees, making the system highly economical for small businesses to adopt, particularly when compared to subscription-based commercial ERP or POS solutions.

2.9.2 Technical Feasibility

The technologies selected for this project – Python for application logic, CustomTkinter and Tkinter for the graphical interface, and SQLite for data storage – are mature, well-documented, and widely supported, with extensive community resources available. The target hardware requirements are modest and compatible with virtually any modern desktop or laptop computer running Windows, ensuring broad technical feasibility for the intended user base.

2.9.3 Operational Feasibility

The system has been designed with a strong focus on usability, featuring a clean, modern, intuitive graphical interface intended to minimize the learning curve for non-technical users such as small shop owners and cashiers. Because the application runs entirely offline and does not require any specialized IT infrastructure or ongoing

maintenance by a dedicated IT team, it is highly operationally feasible for small businesses to adopt and use on a day-to-day basis.

2.9.4 Legal Feasibility

All software components used in this project, including Python, CustomTkinter, Tkinter, and SQLite, are distributed under permissive open-source licenses that allow free use, modification, and distribution, ensuring that the project does not raise any licensing or intellectual property concerns.

2.9.5 Schedule Feasibility

Given the well-defined and modest scope of the project, and the availability of mature, well-documented development tools and libraries, the project timeline outlined in Chapter 13 is considered realistic and achievable within a typical academic project schedule.

Chapter 3

Literature Survey and Requirement Analysis

3.1 Literature Survey

Prior to designing the Smart ERP Billing System, a review of existing academic and industry literature relating to ERP systems, billing automation, and inventory management was carried out. This survey helped identify common design patterns, frequently encountered challenges, and gaps that this project seeks to address. Table 3.1 summarizes the key findings from this literature review.

Author(s)	Year	Method / Approach	Advantages	Limitations
Sharma et al.	2018	Web-based billing system using PHP and MySQL	Accessible from any browser; centralized server storage	Requires constant internet and server hosting; higher deployment cost
Patel and Mehta	2019	Desktop billing application using Java and MySQL	Fast local processing; works offline	Heavyweight JVM dependency; complex installation for end users
Kumar et al.	2019	Android-based mobile billing app	Portable; convenient for very small vendors	Limited screen size restricts reporting features; weaker data security
Singh and Rao	2020	Cloud ERP for SMEs	Centralized access across branches; automatic backups	Recurring subscription cost; dependent on internet availability

Author(s)	Year	Method / Approach	Advantages	Limitations
Verma et al.	2020	Inventory management using barcode scanning	Fast and accurate stock updates	Requires additional barcode hardware; added upfront cost
Das and Bose	2020	Excel-based billing automation using macros	Familiar interface for small business owners	Not scalable; lacks relational integrity and multi-user support
Ahmed et al.	2021	Python and SQLite based point-of-sale system	Lightweight; no server dependency; open-source stack	Limited reporting features; single-user focus
Joshi and Nair	2021	ERP with RFID-based inventory tracking	Real-time, automated stock tracking	High implementation cost; unsuitable for very small businesses
Reddy et al.	2021	Cross-platform billing app using Electron and Node.js	Modern UI; cross-platform support	High memory usage; complex packaging for distribution
Gupta and Iyer	2022	Machine learning based demand forecasting for inventory	Improves reorder accuracy; reduces stockouts	Requires historical data volume; added model complexity
Chowdhury et al.	2022	Cloud-based GST-compliant billing system	Automatic tax compliance updates	Ongoing internet dependency; monthly subscription model
Fernandes and Pinto	2023	Tkinter-based desktop billing tool with SQLite	Free, lightweight, offline-capable	Basic UI compared to modern frameworks; limited built-in reporting
Bhatt and Solanki	2023	Hybrid mobile and desktop ERP with cloud sync	Flexibility across devices	Increased architectural and synchronization complexity

Author(s)	Year	Method / Approach	Advantages	Limitations
Rathi and Menon	2024	CustomTkinter-based modern desktop applications for SMEs	Modern UI with native performance; free and open source	Relatively newer framework with a smaller community than mainstream toolkits

Table 3.1: Literature Survey Summary

3.1.1 Summary of Literature Survey Findings

The literature survey reveals a clear trade-off across existing approaches: web-based and cloud-based systems offer accessibility and centralization but introduce internet dependency and recurring costs, while purely local desktop or mobile applications offer independence and lower cost but often at the expense of advanced reporting or multi-user features. Barcode and RFID-based inventory systems provide excellent accuracy but require additional hardware investment that may not be justified for very small businesses. Notably, few of the reviewed works combine a modern, visually appealing desktop interface with a lightweight, dependency-free embedded database in a manner specifically tuned to the workflow of small retail and service businesses. This gap directly motivates the design choices made in the Smart ERP Billing System, which combines the modern look and feel of CustomTkinter with the simplicity and zero-configuration nature of SQLite, avoiding both the cost of cloud subscriptions and the complexity of additional hardware.

3.2 Requirement Analysis

3.2.1 Functional Requirements

- The system shall allow authorized users to log in using a username and password.
- The system shall allow administrators to create, update, and deactivate user accounts.
- The system shall allow users to add, edit, delete, and search product records in the inventory.

- The system shall automatically alert users when a product's stock falls below a configurable threshold.
- The system shall allow users to add, edit, delete, and search customer records.
- The system shall allow users to create a new bill by selecting products and quantities.
- The system shall automatically calculate the subtotal, applicable tax, discount, and final total for each bill.
- The system shall generate a printable invoice for every completed sale.
- The system shall automatically update inventory stock levels upon completion of a sale.
- The system shall maintain a complete, searchable history of all past invoices.
- The system shall generate sales reports for daily, weekly, and monthly periods.
- The system shall generate inventory reports showing current stock levels and low-stock items.
- The system shall allow configuration of business details, tax rates, and system preferences through a settings module.
- The system shall provide a dashboard summarizing key metrics such as total sales, total products, and low-stock alerts.

3.2.2 Non-Functional Requirements

- **Performance:** The system shall process a typical billing transaction (product lookup, calculation, and invoice generation) in under two seconds.
- **Usability:** The system shall provide an intuitive graphical interface requiring minimal training for non-technical users.
- **Reliability:** The system shall maintain data integrity even in the event of unexpected application closure, through the use of transactional database operations.
- **Security:** The system shall store user passwords in hashed form and shall restrict access to authorized users only.
- **Maintainability:** The system shall be organized using a layered architecture to facilitate future maintenance and feature additions.

- **Portability:** The system shall run on any Windows machine meeting the minimum hardware requirements, without requiring specialized hardware.
- **Scalability:** The system shall efficiently handle a growing volume of products, customers, and transaction history over time.

3.2.3 User Requirements

End users, including cashiers, shop managers, and business owners, require a system that is fast to operate, visually clear, and does not require extensive training. Users need the ability to quickly search for products during billing, view current stock levels at a glance, and generate a professional invoice with minimal steps.

3.2.4 Business Requirements

From a business perspective, the system must reduce the time and cost associated with billing and inventory management, minimize errors that lead to financial loss, and provide business owners with actionable reports that support better purchasing and pricing decisions.

3.2.5 System Requirements

The system requires a Windows-based computer meeting the hardware specifications outlined in Table 2.2, with Python 3.10 or higher installed along with the required third-party libraries, and sufficient disk space to accommodate the growing SQLite database file over time.

Chapter 4

Software Development Life Cycle and System Design

4.1 Software Development Life Cycle Models

4.1.1 Waterfall Model

The Waterfall model is a linear, sequential approach to software development in which each phase – requirement analysis, design, implementation, testing, deployment, and maintenance – must be completed before the next phase begins. It is well suited to projects with clearly defined, stable requirements, but it offers little flexibility to accommodate changes once a phase has been completed.

4.1.2 Agile Model

The Agile model is an iterative approach that breaks development into small, incremental cycles known as sprints, with continuous feedback and re-evaluation of requirements throughout the project. Agile is well suited to projects where requirements may evolve over time, as it allows for frequent reassessment and adjustment.

4.1.3 Spiral Model

The Spiral model combines iterative development with elements of risk analysis, cycling repeatedly through planning, risk analysis, engineering, and evaluation phases. It is particularly well suited to large, complex, high-risk projects where identifying and mitigating risk early is critical.

4.1.4 Prototype Model

The Prototype model involves building an early, simplified working version of the system to gather user feedback before committing to full-scale development. This approach is useful when user requirements are not fully clear at the outset and benefit from early, hands-on feedback.

4.1.5 Selected Model for This Project

For the development of the Smart ERP Billing System, an **Incremental / Iterative model with Agile principles** was selected. This project has a well-understood overall scope – billing, inventory, customer management, and reporting – but individual modules benefit from iterative refinement based on continuous testing and feedback. Developing the system incrementally, module by module (starting with the database schema and authentication, followed by inventory, customer management, billing, and finally reporting), allowed each component to be built, tested, and refined before moving to the next, reducing the risk of major rework late in the project while still maintaining the overall structure and discipline of a well-planned development process.

4.2 System Architecture

The Smart ERP Billing System is built using a **Three-Layer Architecture**, which cleanly separates the concerns of user interface, business logic, and data storage. This separation improves maintainability, testability, and allows each layer to be modified independently without affecting the others.

- **Presentation Layer:** Built using CustomTkinter and Tkinter, this layer is responsible for rendering all screens, capturing user input, and displaying data and feedback to the user. It contains no business logic or direct database access.
- **Business Logic Layer:** Implemented in pure Python, this layer contains all application logic, including validation rules, invoice calculations (subtotal, tax, discount), inventory update logic, and report generation logic. It acts as an intermediary between the presentation layer and the data access layer.
- **Data Access Layer:** Responsible for all direct interaction with the SQLite database, including executing SQL queries, managing database connections, and handling transactions. This layer exposes a clean set of functions (such as `get_product`,

`update_stock`, `create_invoice`) that the business logic layer calls, without needing to know the underlying SQL implementation details.

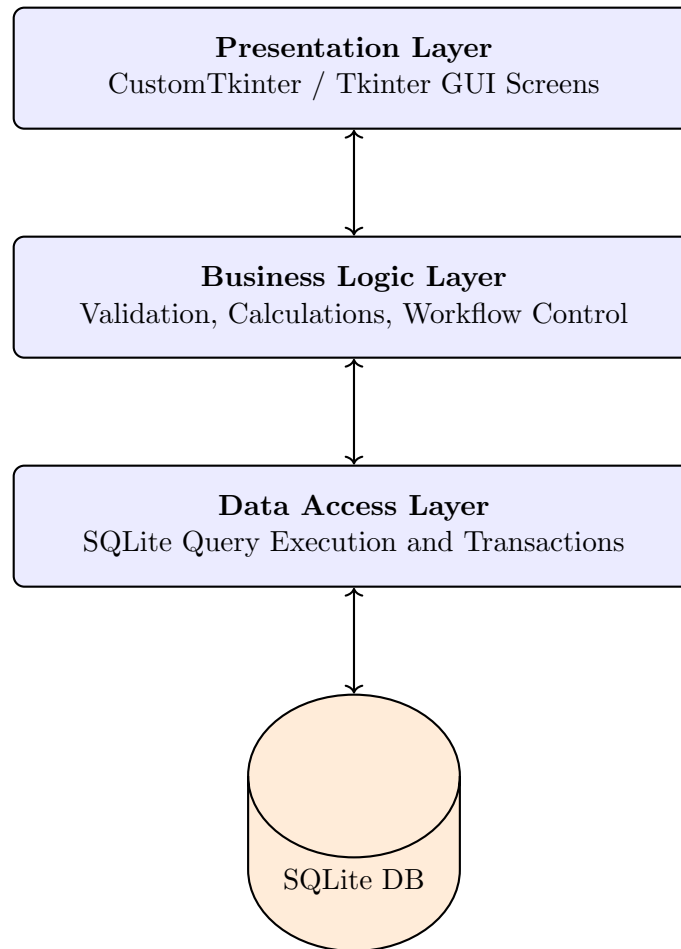


Figure 4.1: Three-Layer Architecture of the Smart ERP Billing System

4.3 Module Diagram

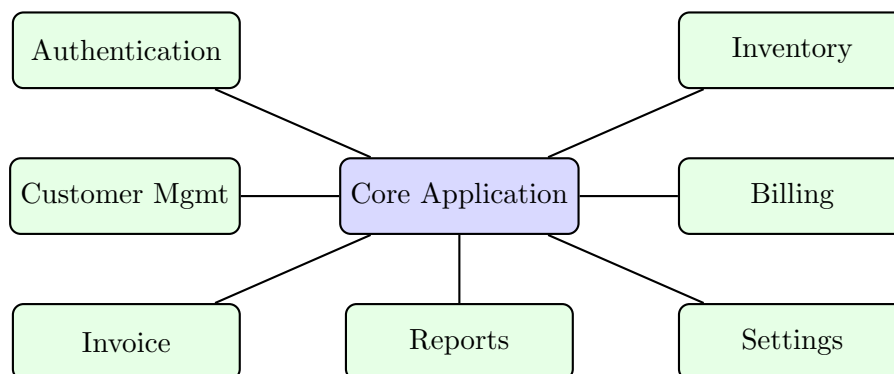


Figure 4.2: Module Diagram of the Smart ERP Billing System

4.4 Workflow Diagram

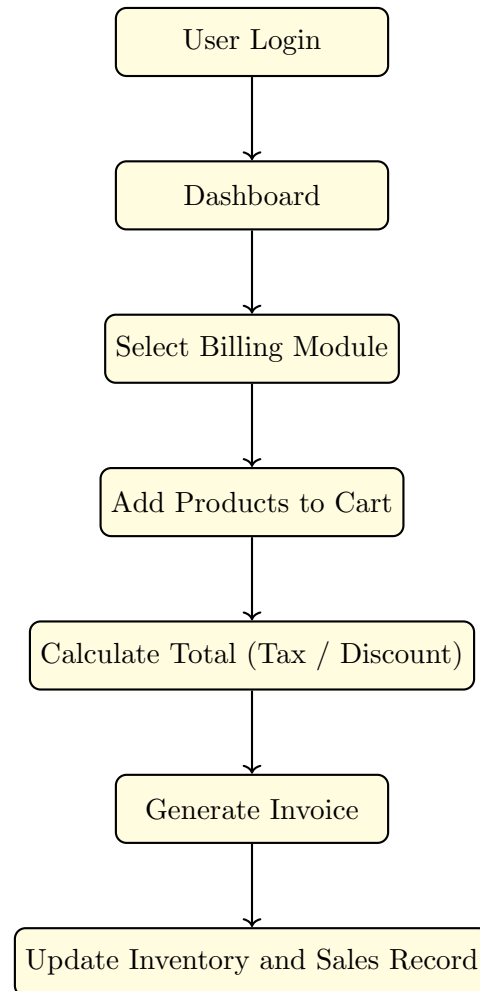


Figure 4.3: High-Level Workflow of a Billing Transaction

4.5 High-Level and Low-Level Design

4.5.1 High-Level Design

At a high level, the system is composed of the presentation, business logic, and data access layers described above, further subdivided into the functional modules shown in Figure 4.2: Authentication, Inventory, Customer Management, Billing, Invoice Generation, Reports, and Settings, all coordinated through a central application controller.

4.5.2 Low-Level Design

At a low level, each module is implemented as a Python package containing a GUI frame class (presentation), a corresponding service class containing business logic functions, and a data access module exposing functions that execute parameterized SQL queries against the SQLite database. For example, the Billing module's service layer exposes functions such as `calculate_invoice_total()`, `apply_discount()`, and `finalize_sale()`, each of which is unit-testable independently of the GUI.

4.6 Database Architecture

The database architecture is centered around a single SQLite database file that stores all application data in a set of normalized relational tables, described in detail in Chapter 6. SQLite was selected for its zero-configuration, serverless nature, its full support for standard SQL and ACID-compliant transactions, and its excellent performance characteristics for single-user desktop applications of this scale.

4.7 Component Diagram

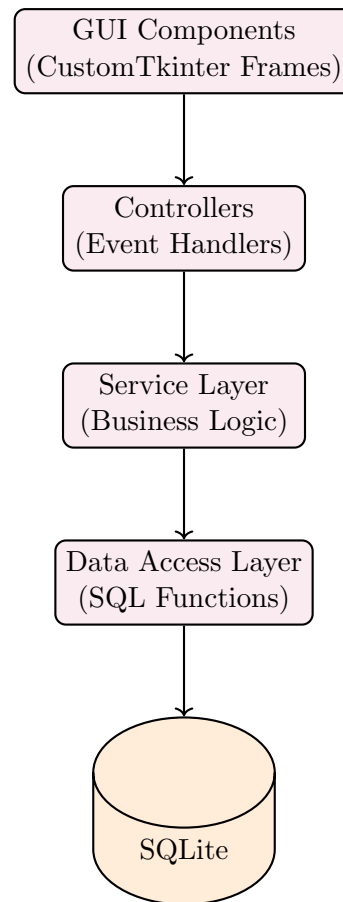


Figure 4.4: Component Diagram

4.8 Deployment Diagram

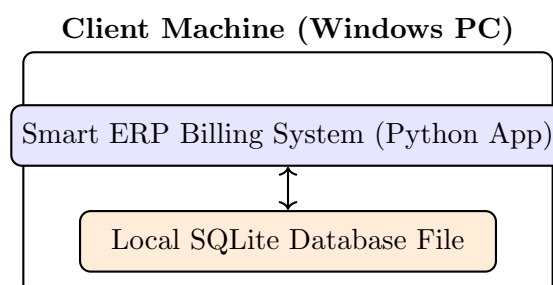


Figure 4.5: Deployment Diagram (Single Machine Deployment)

Chapter 5

System Modeling: DFD and UML Diagrams

5.1 Data Flow Diagrams

5.1.1 Context Diagram (Level 0)

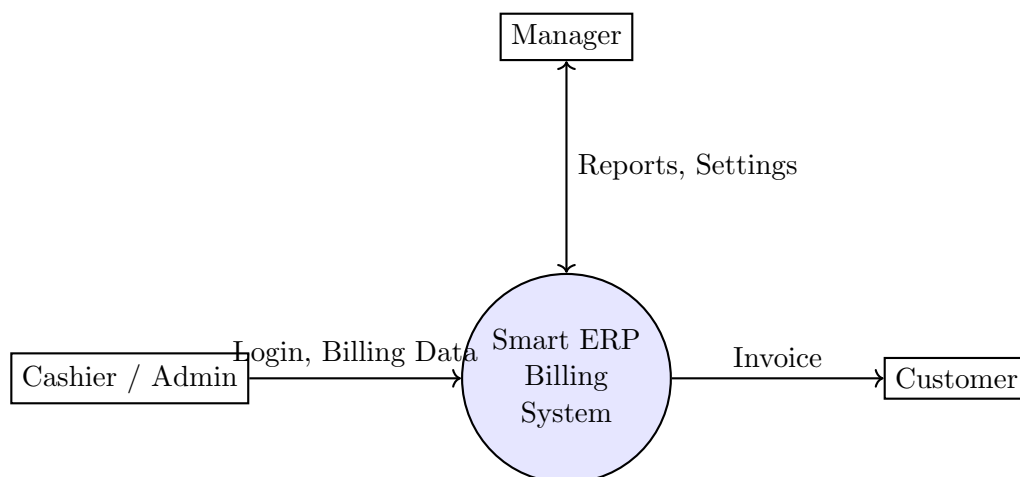


Figure 5.1: Context Level (Level 0) Data Flow Diagram

5.1.2 Level 1 DFD

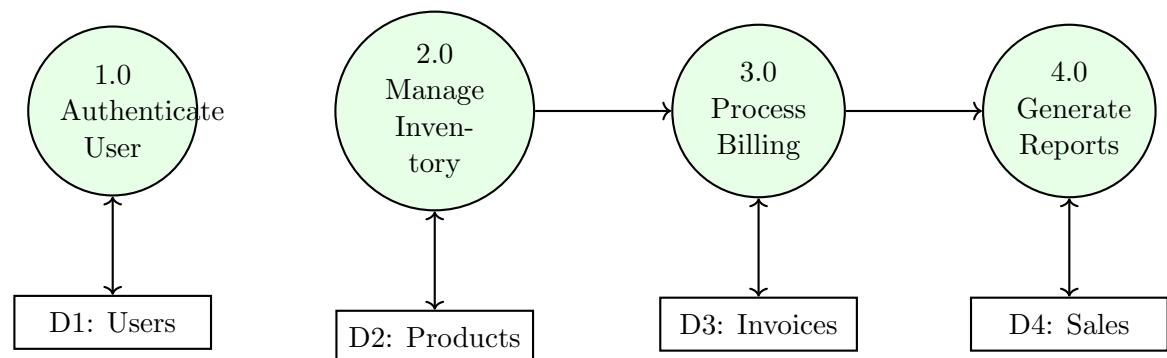


Figure 5.2: Level 1 Data Flow Diagram

5.1.3 Level 2 DFD: Billing Process

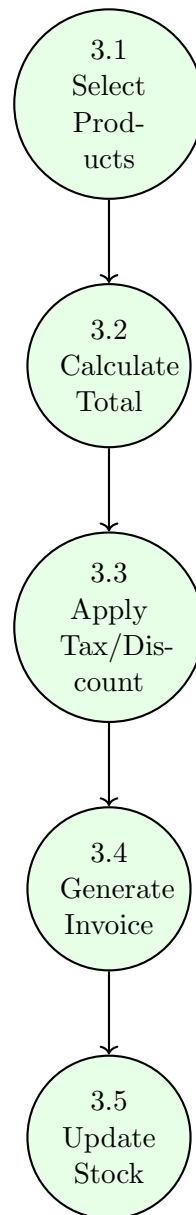


Figure 5.3: Level 2 Data Flow Diagram (Billing Sub-Process)

5.2 Use Case Diagram

Key actors of the system include the **Admin**, who has full system access including user and settings management; the **Cashier**, who primarily performs billing operations; the **Manager**, who reviews reports and inventory; and the **Customer**, who indirectly interacts with the system through receiving invoices.

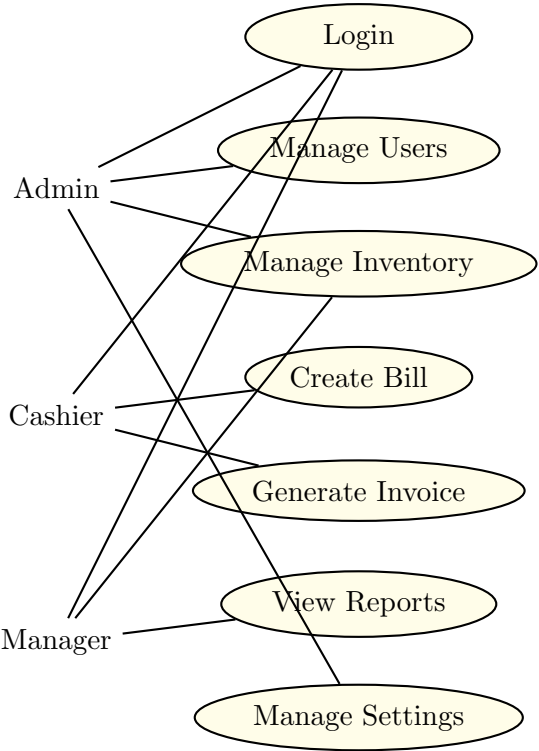


Figure 5.4: Use Case Diagram

5.3 Sequence Diagram: Billing Process

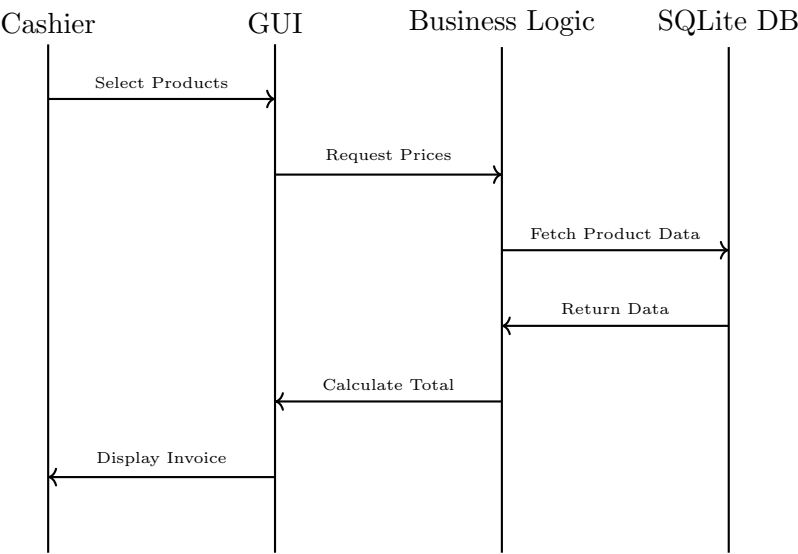


Figure 5.5: Sequence Diagram for Billing Process

5.4 Activity Diagram: Billing Process

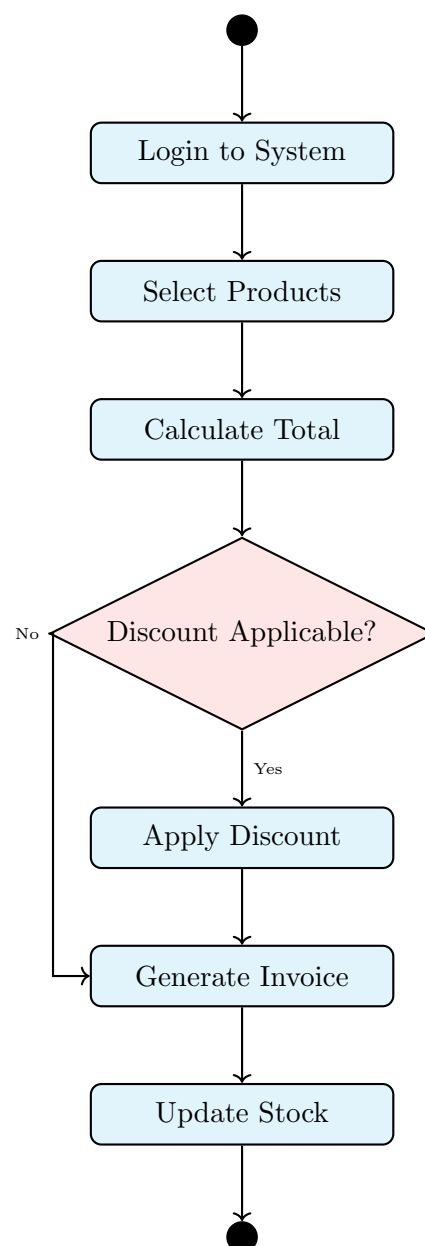


Figure 5.6: Activity Diagram for Billing Process

5.5 Class Diagram

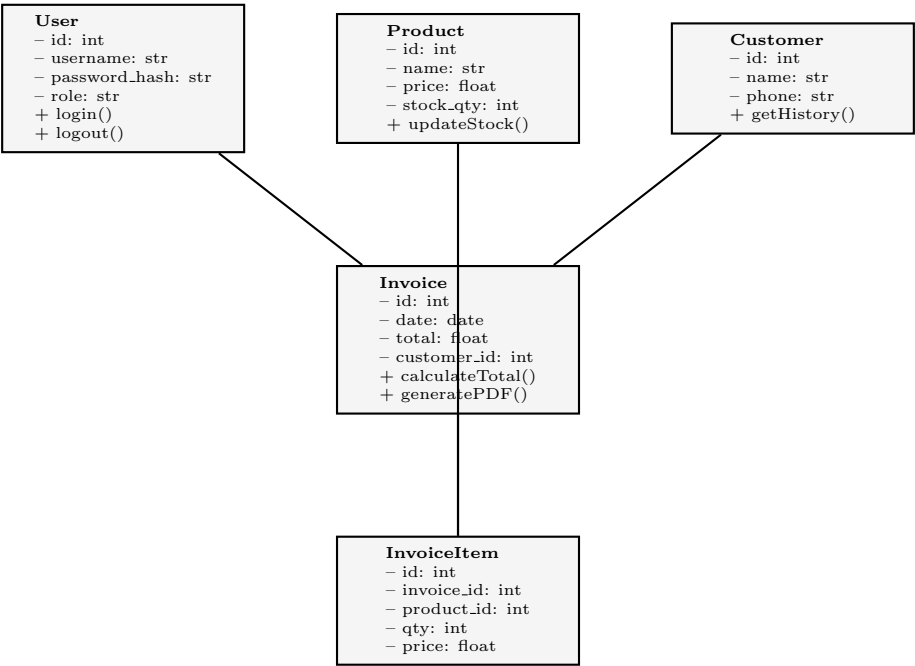


Figure 5.7: Class Diagram

5.6 State Diagram: Invoice Lifecycle

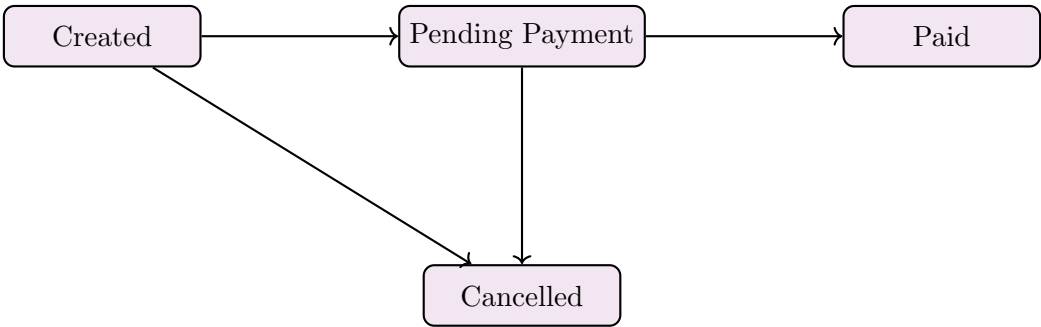


Figure 5.8: State Diagram for Invoice Lifecycle

5.7 Entity Relationship Diagram

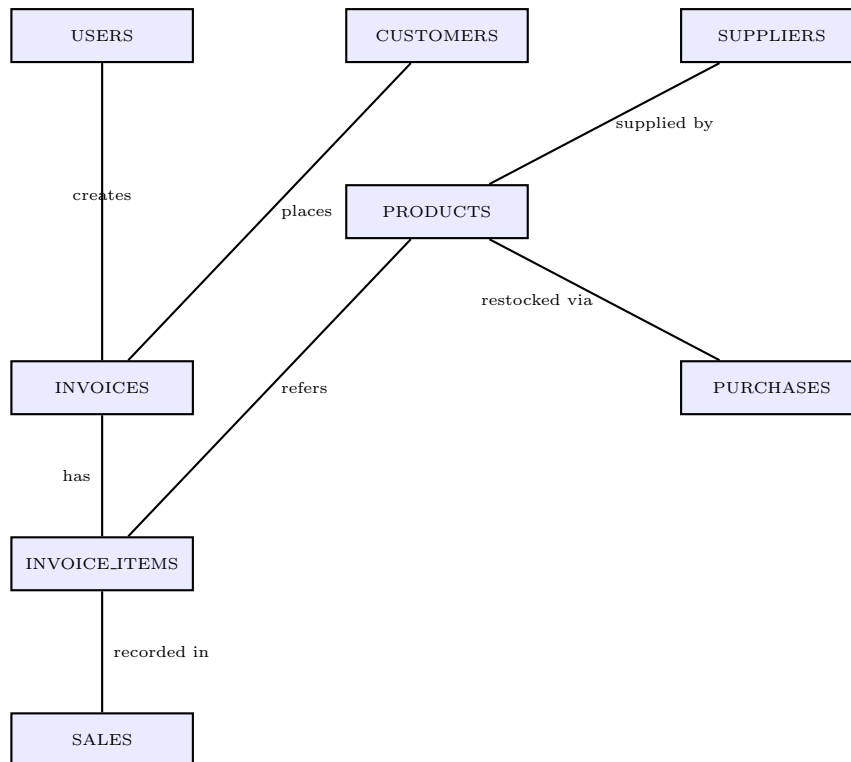


Figure 5.9: Entity Relationship Diagram

Chapter 6

Database Design

6.1 Overview

The Smart ERP Billing System uses a relational database implemented in SQLite. The schema has been designed following standard normalization principles (up to Third Normal Form) to eliminate data redundancy and maintain referential integrity across all core business entities: users, customers, products, suppliers, purchases, invoices, invoice items, sales, and settings.

6.2 Users Table

Table 6.1: Users Table Structure

Column	Type	Constraint	Description
user_id	INTEGER	PRIMARY KEY	Unique identifier for each user
username	TEXT	UNIQUE, NOT NULL	Login username
password_hash	TEXT	NOT NULL	Securely hashed password
role	TEXT	NOT NULL	Admin, Cashier, or Manager
full_name	TEXT	NOT NULL	Full name of the user
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation date
is_active	INTEGER	DEFAULT 1	Account active status flag

6.3 Customers Table

Table 6.2: Customers Table Structure

Column	Type	Constraint	Description
customer_id	INTEGER	PRIMARY KEY	Unique identifier for each customer
name	TEXT	NOT NULL	Customer full name
phone	TEXT	UNIQUE	Contact phone number
email	TEXT		Customer email address
address	TEXT		Customer address
credit_balance	REAL	DEFAULT 0	Outstanding due balance
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation date

6.4 Products Table

Table 6.3: Products Table Structure

Column	Type	Constraint	Description
product_id	INTEGER	PRIMARY KEY	Unique identifier for each product
name	TEXT	NOT NULL	Product name
category	TEXT		Product category
price	REAL	NOT NULL	Selling price per unit
cost_price	REAL		Purchase cost per unit
stock_qty	INTEGER	DEFAULT 0	Current stock quantity
reorder_level	INTEGER	DEFAULT 5	Minimum threshold for low-stock alert
supplier_id	INTEGER	FOREIGN KEY	References Suppliers table

6.5 Invoices Table

Table 6.4: Invoices Table Structure

Column	Type	Constraint	Description
invoice_id	INTEGER	PRIMARY KEY	Unique identifier for each invoice
customer_id	INTEGER	FOREIGN KEY	References Customers table
user_id	INTEGER	FOREIGN KEY	References Users table (cashier)
invoice_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date and time of billing
subtotal	REAL	NOT NULL	Sum before tax/discount
tax_amount	REAL	DEFAULT 0	Total tax applied
discount_amount	REAL	DEFAULT 0	Total discount applied
total_amount	REAL	NOT NULL	Final payable amount
payment_status	TEXT	DEFAULT 'Paid'	Paid, Pending, or Cancelled

6.6 InvoiceItems Table

Table 6.5: InvoiceItems Table Structure

Column	Type	Constraint	Description
item_id	INTEGER	PRIMARY KEY	Unique identifier for each line item
invoice_id	INTEGER	FOREIGN KEY	References Invoices table
product_id	INTEGER	FOREIGN KEY	References Products table
quantity	INTEGER	NOT NULL	Quantity sold
unit_price	REAL	NOT NULL	Price at time of sale
line_total	REAL	NOT NULL	$\text{quantity} \times \text{unit_price}$

6.7 Suppliers, Purchases, Sales, Reports and Settings Tables

Table 6.6: Suppliers Table Structure

Column	Type	Constraint	Description
supplier_id	INTEGER	PRIMARY KEY	Unique identifier for each supplier
name	TEXT	NOT NULL	Supplier name
phone	TEXT		Contact number
address	TEXT		Supplier address

Table 6.7: Purchases Table Structure

Column	Type	Constraint	Description
purchase_id	INTEGER	PRIMARY KEY	Unique identifier for each purchase
supplier_id	INTEGER	FOREIGN KEY	References Suppliers table
product_id	INTEGER	FOREIGN KEY	References Products table
quantity	INTEGER	NOT NULL	Quantity purchased
purchase_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date of purchase

Table 6.8: Sales Summary Table Structure

Column	Type	Constraint	Description
sale_id	INTEGER	PRIMARY KEY	Unique identifier
invoice_id	INTEGER	FOREIGN KEY	References Invoices table
sale_date	DATE	NOT NULL	Date of sale (used for reporting)
total_amount	REAL	NOT NULL	Amount for that sale

Table 6.9: Settings Table Structure

Column	Type	Constraint	Description
setting_id	INTEGER	PRIMARY KEY	Unique identifier
business_name	TEXT		Business name for invoices
tax_rate	REAL	DEFAULT 0	Default tax percentage
currency_symbol	TEXT	DEFAULT '\$'	Currency symbol used

6.8 Primary Keys, Foreign Keys and Relationships

Every table in the schema has a single-column integer primary key that uniquely identifies each record. Referential integrity is enforced through foreign key constraints: `Products.supplier_id` references `Suppliers.supplier_id`; `Invoices.customer_id` references `Customers.customer_id`; `Invoices.user_id` references `Users.user_id`; `InvoiceItems.invoice_id` references `Invoices.invoice_id`; `InvoiceItems.product_id` references `Products.product_id`; and `Purchases.supplier_id` and `Purchases.product_id` reference their respective parent tables. This design establishes a clear one-to-many relationship pattern throughout the schema: one customer can have many invoices, one invoice can have many invoice items, and one product can appear in many invoice items and purchases.

6.9 Normalization

The database schema has been normalized to Third Normal Form (3NF). Each table stores information about a single entity type, all non-key attributes are fully functionally dependent on the primary key, and there are no transitive dependencies between non-key attributes. For example, rather than storing product name and price directly within the `InvoiceItems` table, only the `product_id` is stored, with the actual product name and current price retrieved by joining with the `Products` table; the `unit_price` is nonetheless separately recorded in `InvoiceItems` to preserve the historical price at the time of sale, since product prices may change over time.

6.10 Sample Table Creation SQL

```
1 CREATE TABLE users (  
2     user_id INTEGER PRIMARY KEY AUTOINCREMENT,  
3     username TEXT UNIQUE NOT NULL,  
4     password_hash TEXT NOT NULL,  
5     role TEXT NOT NULL,  
6     full_name TEXT NOT NULL,  
7     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
8     is_active INTEGER DEFAULT 1  
9 );  
10  
11 CREATE TABLE products (  
12     product_id INTEGER PRIMARY KEY AUTOINCREMENT,  
13     name TEXT NOT NULL,
```

```
14     category TEXT,
15     price REAL NOT NULL,
16     cost_price REAL,
17     stock_qty INTEGER DEFAULT 0,
18     reorder_level INTEGER DEFAULT 5,
19     supplier_id INTEGER,
20     FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id)
21 );
22
23 CREATE TABLE invoices (
24     invoice_id INTEGER PRIMARY KEY AUTOINCREMENT,
25     customer_id INTEGER,
26     user_id INTEGER NOT NULL,
27     invoice_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
28     subtotal REAL NOT NULL,
29     tax_amount REAL DEFAULT 0,
30     discount_amount REAL DEFAULT 0,
31     total_amount REAL NOT NULL,
32     payment_status TEXT DEFAULT 'Paid',
33     FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
34     FOREIGN KEY (user_id) REFERENCES users(user_id)
35 );
36
37 CREATE TABLE invoice_items (
38     item_id INTEGER PRIMARY KEY AUTOINCREMENT,
39     invoice_id INTEGER NOT NULL,
40     product_id INTEGER NOT NULL,
41     quantity INTEGER NOT NULL,
42     unit_price REAL NOT NULL,
43     line_total REAL NOT NULL,
44     FOREIGN KEY (invoice_id) REFERENCES invoices(invoice_id),
45     FOREIGN KEY (product_id) REFERENCES products(product_id)
46 );
```

Listing 6.1: SQL Schema Definition for Core Tables

Chapter 7

GUI Design and Module Explanation

7.1 GUI Design Philosophy

The graphical interface of the Smart ERP Billing System is built using **CustomTkinter**, a modern extension of the standard Tkinter toolkit that provides rounded corners, dark and light theme support, and a visually contemporary appearance while retaining the simplicity and native performance of Tkinter. The design philosophy emphasizes clarity, minimal clutter, and consistent navigation, ensuring that users with limited computer experience can operate the system confidently after only a brief orientation.

7.2 Login Screen

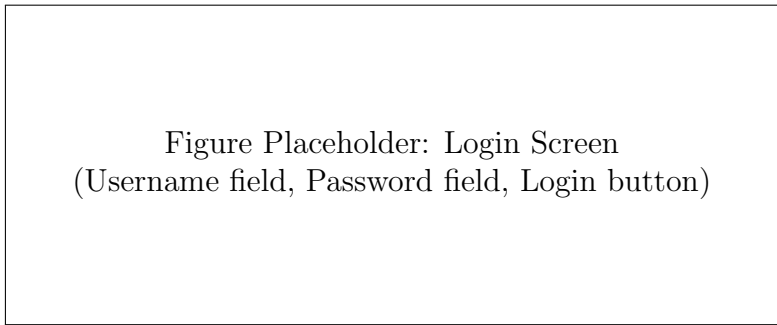


Figure Placeholder: Login Screen
(Username field, Password field, Login button)

Figure 7.1: Login Screen

The login screen is the entry point of the application. It presents fields for username and password, along with a login button. Upon submission, the entered password is hashed and compared against the stored hash for the given username; successful authentication redirects the user to the dashboard, while failed attempts display a

clear, non-technical error message.

7.3 Dashboard

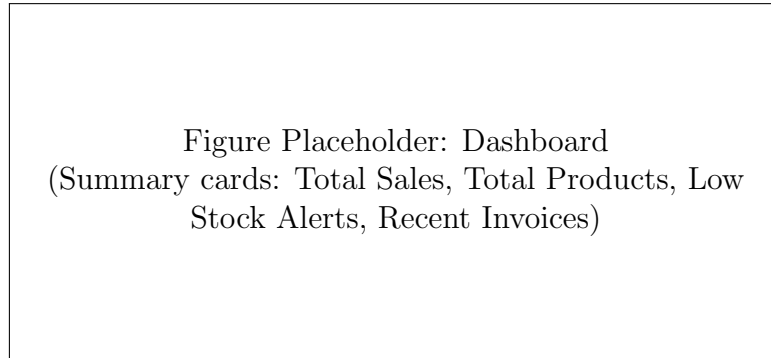


Figure 7.2: Dashboard Screen

The dashboard presents an at-a-glance summary of key business metrics, including total sales for the current day, total number of products in inventory, count of products below their reorder threshold, and a list of the most recent invoices. Navigation buttons on the side panel allow quick access to all other modules.

7.4 Inventory Screen

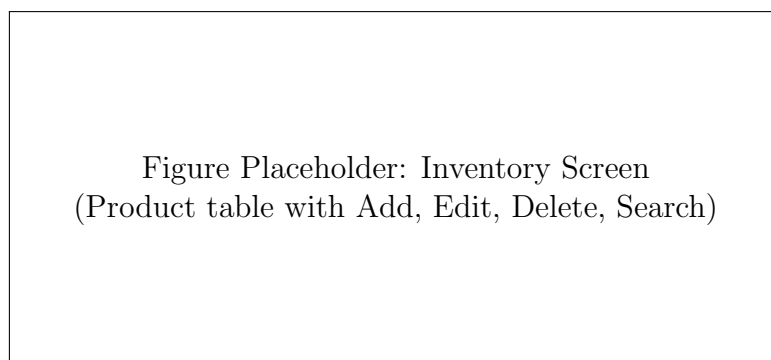


Figure 7.3: Inventory Management Screen

The inventory screen displays a searchable, sortable table of all products, showing name, category, price, and current stock quantity, with rows for low-stock products visually highlighted. Buttons allow the user to add a new product, edit an existing product's details, or remove a discontinued product.

7.5 Customer Screen

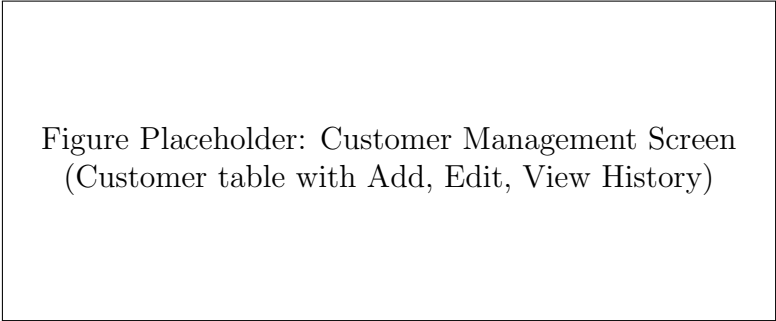


Figure Placeholder: Customer Management Screen
(Customer table with Add, Edit, View History)

Figure 7.4: Customer Management Screen

The customer screen provides a searchable list of all registered customers, along with the ability to add new customers, edit existing customer details, and view a customer's complete purchase history and any outstanding credit balance.

7.6 Billing Screen

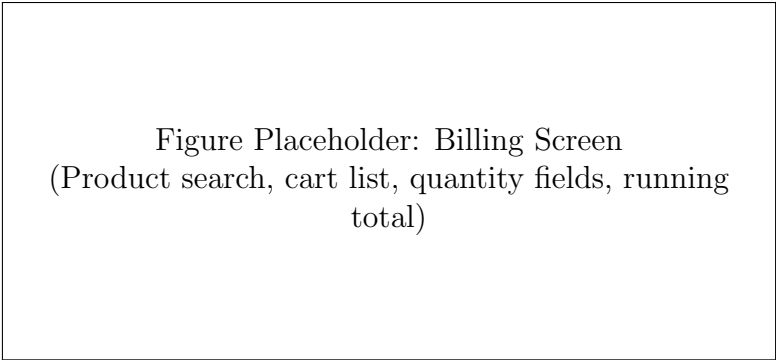


Figure Placeholder: Billing Screen
(Product search, cart list, quantity fields, running total)

Figure 7.5: Billing Screen

The billing screen is the most frequently used part of the application. It allows the cashier to search for and select products, specify quantities, and view a running subtotal that updates automatically as items are added or removed from the cart. Tax and discount fields allow adjustments before the sale is finalized.

7.7 Invoice Screen

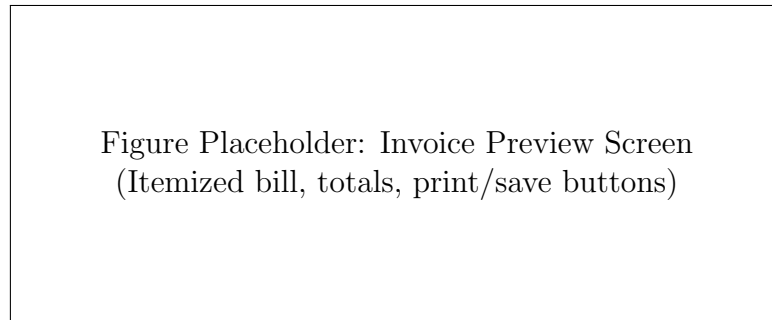


Figure 7.6: Invoice Preview Screen

After a sale is finalized, the invoice screen displays a formatted, itemized preview of the invoice, including business details, line items, subtotal, tax, discount, and grand total, with options to print the invoice or save it as a PDF file.

7.8 Reports Screen

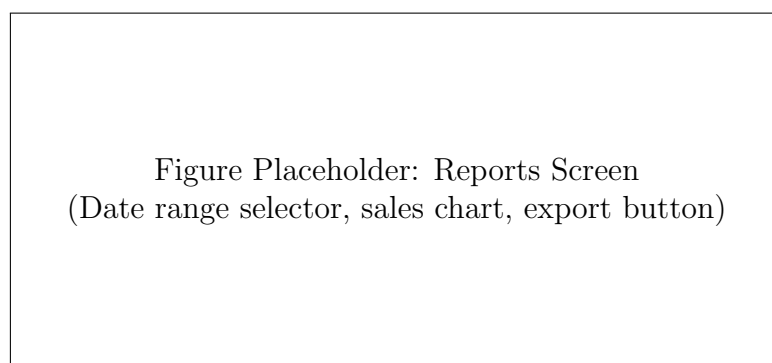


Figure 7.7: Reports Screen

The reports screen allows users to generate sales and inventory reports for a selected date range, presented both as summary tables and simple charts, with the option to export the report to a file for offline analysis or record keeping.

7.9 Settings Screen

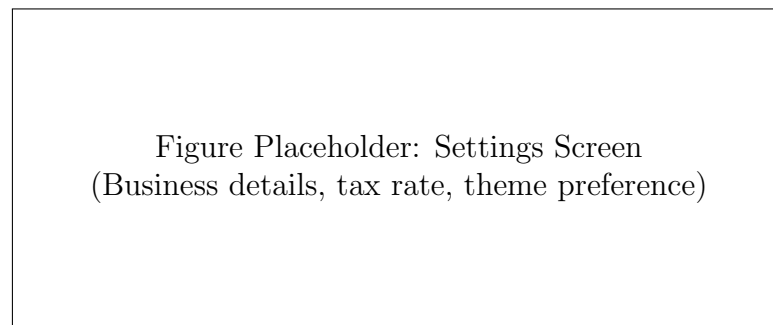


Figure 7.8: Settings Screen

The settings screen allows administrators to configure business details (such as the business name displayed on invoices), the default tax rate, currency symbol, and interface theme preference (light or dark mode).

7.10 Profile and Logout

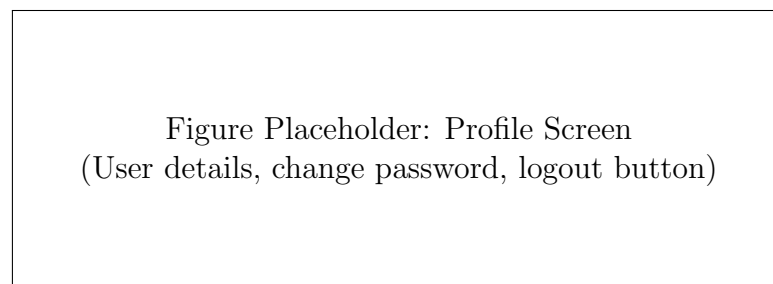


Figure 7.9: Profile Screen

The profile screen allows the logged-in user to view their account details, change their password, and safely log out of the application, returning to the login screen.

7.11 Complete Module Explanation

7.11.1 Authentication Module

Handles user login, password hashing and verification, session management, and role-based access control, ensuring that only authorized users can access sensitive functionality such as user management and settings.

7.11.2 Inventory Module

Manages all product-related operations, including adding new products, editing existing product details, deleting discontinued products, searching and filtering the product catalog, and monitoring stock levels against configurable reorder thresholds.

7.11.3 Customer Module

Manages customer records, including adding and editing customer profiles, searching the customer database, and retrieving a given customer's complete purchase history and outstanding balance.

7.11.4 Billing Module

Coordinates the process of creating a new sale: selecting products, specifying quantities, calculating running totals, applying tax and discounts, and finalizing the transaction, which in turn triggers invoice generation and inventory updates.

7.11.5 Invoice Module

Responsible for formatting and generating a professional, itemized invoice document for each completed sale, including business details, line items, and totals, with support for printing or saving as a PDF file.

7.11.6 Sales Module

Records every completed sale into the sales history, providing the underlying data used by the reporting module to generate daily, weekly, and monthly summaries.

7.11.7 Reports Module

Aggregates sales and inventory data to generate reports such as total sales over a given period, best-selling products, and current low-stock items, presenting this information in both tabular and simple chart form.

7.11.8 Settings Module

Allows configuration of system-wide parameters such as business name, default tax rate, currency symbol, and interface theme, all of which are persisted in the Settings table of the database.

7.11.9 Database Module

Serves as the data access layer of the application, managing the SQLite connection, executing parameterized queries, and handling transactions to ensure that all database operations are performed safely and consistently.

Chapter 8

Algorithms and Mathematical Model

8.1 Algorithms Used

8.1.1 Product Search Algorithm

Product search within the billing and inventory screens is implemented using a linear scan combined with SQLite’s built-in indexed `LIKE` query, allowing partial, case-insensitive matches on product name or code. For the typical product catalog size of a small business (a few hundred to a few thousand products), this approach provides sub-second response times without the added complexity of a dedicated search engine.

Algorithm 1 Product Search

```

1: procedure SEARCHPRODUCT(query)
2:   results  $\leftarrow$  EXECUTESQL("SELECT * FROM products WHERE name LIKE
   ?" , %query%)
3:   return results
4: end procedure
  
```

8.1.2 Sorting Algorithm

Product and report listings are sorted using Python’s built-in `sorted()` function (which implements the Timsort algorithm), providing efficient $O(n \log n)$ sorting by any selected column, such as name, price, or stock quantity.

8.1.3 SQLite Query Optimization

Frequently queried columns, such as `product_id`, `customer_id`, and `invoice_date`, are indexed to ensure fast lookups even as the volume of historical data grows. Parameterized queries are used throughout to prevent SQL injection and to allow SQLite’s query planner to cache execution plans.

8.1.4 Invoice Calculation Algorithm

Algorithm 2 Invoice Total Calculation

```

1: procedure CALCULATEINVOICE(cartItems, taxRate, discountRate)
2:   subtotal  $\leftarrow$  0
3:   for each item in cartItems do
4:     subtotal  $\leftarrow$  subtotal + (item.price  $\times$  item.quantity)
5:   end for
6:   taxAmount  $\leftarrow$  subtotal  $\times$  taxRate
7:   discountAmount  $\leftarrow$  subtotal  $\times$  discountRate
8:   total  $\leftarrow$  subtotal + taxAmount - discountAmount
9:   return subtotal, taxAmount, discountAmount, total
10: end procedure

```

8.1.5 Tax Calculation

Tax is calculated as a configurable percentage of the subtotal, as defined in the Settings module, allowing the business owner to adjust the tax rate to match local regulatory requirements without modifying application code.

8.1.6 Discount Calculation

Discounts may be applied either as a flat percentage of the subtotal or as a fixed monetary amount, depending on business policy, with the final discount amount always validated to ensure it does not exceed the subtotal.

8.1.7 Stock Update Algorithm

Algorithm 3 Stock Update After Sale

```

1: procedure UPDATESTOCK(cartItems)
2:   for each item in cartItems do
3:     product  $\leftarrow$  GETPRODUCT(item.product_id)
4:     product.stock_qty  $\leftarrow$  product.stock_qty - item.quantity
5:     SAVEPRODUCT(product)
6:     if product.stock_qty  $\leq$  product.reorder_level then
7:       RAISELOWSTOCKALERT(product)
8:     end if
9:   end for
10: end procedure

```

8.2 Mathematical Model

8.2.1 Input

The mathematical model of the billing process takes as input a set of cart items $I = \{i_1, i_2, \dots, i_n\}$, where each item i_k has an associated quantity q_k and unit price p_k , along with a tax rate t and a discount rate d .

8.2.2 Process

The subtotal S is computed as:

$$S = \sum_{k=1}^n q_k \times p_k \quad (8.1)$$

The tax amount T is computed as:

$$T = S \times t \quad (8.2)$$

The discount amount D is computed as:

$$D = S \times d \quad (8.3)$$

8.2.3 Output

The final total F payable by the customer is given by:

$$F = S + T - D \quad (8.4)$$

8.2.4 Constraints

$$0 \leq d \leq 1, \quad 0 \leq t, \quad D \leq S, \quad F \geq 0 \quad (8.5)$$

These constraints ensure that the discount rate remains within a valid percentage range, that the discount amount never exceeds the subtotal, and that the final payable amount is never negative, protecting both the business and the customer from invalid billing outcomes.

Chapter 9

Implementation

9.1 Implementation Overview

The implementation of the Smart ERP Billing System follows the Three-Layer Architecture described in Chapter 4. Each functional module – authentication, inventory, customer management, billing, invoicing, reporting, and settings – is implemented as a self-contained Python package, with a clear separation between GUI code, business logic, and database access code. This section explains the implementation of each module in detail, along with representative code excerpts.

9.2 Project Folder Structure

```
1 SmartERPBillingSystem/  
2 |-- main.py  
3 |-- database/  
4 |   |-- db_connection.py  
5 |   |-- schema.sql  
6 |-- modules/  
7 |   |-- auth/  
8 |       |-- auth_gui.py  
9 |       |-- auth_service.py  
10 |   |-- inventory/  
11 |       |-- inventory_gui.py  
12 |       |-- inventory_service.py  
13 |   |-- customer/  
14 |       |-- customer_gui.py  
15 |       |-- customer_service.py  
16 |   |-- billing/  
17 |       |-- billing_gui.py  
18 |       |-- billing_service.py  
19 |   |-- invoice/  
20 |       |-- invoice_generator.py  
21 |   |-- reports/
```

```
22 | | | -- reports_gui.py
23 | | | -- reports_service.py
24 | | -- settings/
25 | | | -- settings_gui.py
26 | | | -- settings_service.py
27 | -- assets/
28 | | -- icons/
29 | | -- logo.png
30 | -- requirements.txt
31 | -- README.md
```

Listing 9.1: Project Folder Structure

9.3 Database Connection Module

The `db_connection.py` module centralizes all SQLite connection handling, ensuring that a single, properly configured connection object is reused across the application and that foreign key constraints are enforced.

```
1 import sqlite3
2
3 class DatabaseConnection:
4     """Handles the SQLite database connection for the application."""
5
6     _instance = None
7
8     def __new__(cls, db_path="erp_billing.db"):
9         if cls._instance is None:
10             cls._instance = super().__new__(cls)
11             cls._instance.conn = sqlite3.connect(db_path,
12 check_same_thread=False)
13             cls._instance.conn.execute("PRAGMA foreign_keys = ON")
14             cls._instance.conn.row_factory = sqlite3.Row
15             return cls._instance
16
17     def get_cursor(self):
18         return self.conn.cursor()
19
20     def commit(self):
21         self.conn.commit()
22
23     def close(self):
24         self.conn.close()
```

Listing 9.2: Database Connection Setup

The `DatabaseConnection` class is implemented using the Singleton design pattern, ensuring that only one active connection to the SQLite database exists throughout the application's lifetime, which avoids the overhead of repeatedly opening and closing connections and prevents database locking issues.

9.4 Authentication Module Implementation

```
1 import hashlib
2 import os
3
4 class AuthService:
5     """Handles user authentication and password security."""
6
7     def __init__(self, db):
8         self.db = db
9
10    def hash_password(self, password, salt=None):
11        if salt is None:
12            salt = os.urandom(16).hex()
13        hashed = hashlib.sha256((salt + password).encode()).hexdigest()
14        return f"{salt}${hashed}"
15
16    def verify_password(self, password, stored_hash):
17        salt, hashed = stored_hash.split("$")
18        return self.hash_password(password, salt) == stored_hash
19
20    def login(self, username, password):
21        cursor = self.db.get_cursor()
22        cursor.execute(
23            "SELECT * FROM users WHERE username = ? AND is_active = 1",
24            (username,)
25        )
26        user = cursor.fetchone()
27        if user and self.verify_password(password, user["password_hash"]):
28            return user
29        return None
```

Listing 9.3: Authentication Service

The `AuthService` class implements password hashing using SHA-256 combined with a randomly generated salt for each user, which protects against common attacks such as rainbow table lookups. The `login()` method retrieves the user record by username

and verifies the supplied password against the stored salted hash before granting access.

9.5 Inventory Module Implementation

```

1 class InventoryService:
2     """Handles all inventory-related business logic."""
3
4     def __init__(self, db):
5         self.db = db
6
7     def add_product(self, name, category, price, cost_price,
8 stock_qty, reorder_level):
9         cursor = self.db.get_cursor()
10        cursor.execute(
11            """INSERT INTO products
12                (name, category, price, cost_price, stock_qty,
13 reorder_level)
14                VALUES (?, ?, ?, ?, ?, ?)""",
15            (name, category, price, cost_price, stock_qty,
16 reorder_level)
17        )
18        self.db.commit()
19        return cursor.lastrowid
20
21    def search_products(self, keyword):
22        cursor = self.db.get_cursor()
23        cursor.execute(
24            "SELECT * FROM products WHERE name LIKE ?",
25            (f"%{keyword}%",)
26        )
27        return cursor.fetchall()
28
29    def get_low_stock_products(self):
30        cursor = self.db.get_cursor()
31        cursor.execute(
32            "SELECT * FROM products WHERE stock_qty <= reorder_level"
33        )
34        return cursor.fetchall()
35
36    def update_stock(self, product_id, quantity_change):
37        cursor = self.db.get_cursor()
38        cursor.execute(
39            "UPDATE products SET stock_qty = stock_qty + ? WHERE
40 product_id = ?",

```

```
37         (quantity_change, product_id)
38     )
39     self.db.commit()
```

Listing 9.4: Inventory Service

9.6 Billing Module Implementation

```
1 class BillingService:
2     """Handles billing calculations and invoice finalization."""
3
4     def __init__(self, db, inventory_service):
5         self.db = db
6         self.inventory_service = inventory_service
7
8     def calculate_totals(self, cart_items, tax_rate=0.0,
9 discount_rate=0.0):
10         subtotal = sum(item["price"] * item["quantity"] for item in
11 cart_items)
12         tax_amount = subtotal * tax_rate
13         discount_amount = subtotal * discount_rate
14         total = subtotal + tax_amount - discount_amount
15         return {
16             "subtotal": round(subtotal, 2),
17             "tax_amount": round(tax_amount, 2),
18             "discount_amount": round(discount_amount, 2),
19             "total": round(total, 2),
20         }
21
22     def finalize_sale(self, cart_items, customer_id, user_id,
23 tax_rate, discount_rate):
24         totals = self.calculate_totals(cart_items, tax_rate,
25 discount_rate)
26         cursor = self.db.get_cursor()
27         cursor.execute(
28             """INSERT INTO invoices
29                 (customer_id, user_id, subtotal, tax_amount,
30 discount_amount, total_amount)
31                 VALUES (?, ?, ?, ?, ?, ?)""",
32             (customer_id, user_id, totals["subtotal"], totals["
33 tax_amount"],
34             totals["discount_amount"], totals["total"])
35         )
36         invoice_id = cursor.lastrowid
```

```

32     for item in cart_items:
33         line_total = round(item["price"] * item["quantity"], 2)
34         cursor.execute(
35             """INSERT INTO invoice_items
36             (invoice_id, product_id, quantity, unit_price,
37             line_total)
38             VALUES (?, ?, ?, ?, ?)""",
39             (invoice_id, item["product_id"], item["quantity"],
40             item["price"], line_total)
41         )
42         self.inventory_service.update_stock(item["product_id"], -
43         item["quantity"])
44     self.db.commit()
45     return invoice_id, totals

```

Listing 9.5: Billing Service

The `finalize_sale()` method demonstrates the coordination between the billing and inventory modules: after calculating and recording the invoice and its line items, it calls `update_stock()` with a negative quantity change for each sold item, ensuring that inventory levels are decremented atomically as part of the same overall transaction.

9.7 GUI Implementation Example

```

1  import customtkinter as ctk
2
3  class LoginFrame(ctk.CTkFrame):
4      def __init__(self, master, auth_service, on_success):
5          super().__init__(master)
6          self.auth_service = auth_service
7          self.on_success = on_success
8
9          ctk.CTkLabel(self, text="Smart ERP Billing System",
10                      font=("Segoe UI", 22, "bold")).pack(pady=(30,
11                      10))
12
13          self.username_entry = ctk.CTkEntry(self, placeholder_text="
14          Username", width=260)
15          self.username_entry.pack(pady=10)
16
17          self.password_entry = ctk.CTkEntry(self, placeholder_text="
18          Password",
19                                              show="*", width=260)

```

```
17         self.password_entry.pack(pady=10)
18
19         self.error_label = ctk.CTkLabel(self, text="", text_color="
red")
20         self.error_label.pack(pady=5)
21
22         ctk.CTkButton(self, text="Login", command=self.handle_login,
23                       width=260).pack(pady=20)
24
25     def handle_login(self):
26         username = self.username_entry.get()
27         password = self.password_entry.get()
28         user = self.auth_service.login(username, password)
29         if user:
30             self.on_success(user)
31         else:
32             self.error_label.configure(text="Invalid username or
password")
```

Listing 9.6: CustomTkinter Login Screen (Simplified)

9.8 Event Handling

Event handling in the GUI layer follows the standard Tkinter/CustomTkinter callback pattern, where widget actions such as button clicks are bound to controller methods (for example, `command=self.handle_login`) that validate input, invoke the appropriate service layer method, and update the interface based on the result, without embedding any direct database access code inside the GUI layer itself.

9.9 Invoice Generation

Invoice generation is handled by a dedicated `invoice_generator.py` module, which formats invoice data retrieved from the database into a printable PDF document using the `reportlab` library, including the business header, itemized product table, and calculated totals, ready for printing or saving to disk.

Chapter 10

Testing

10.1 Testing Overview

A comprehensive testing strategy was adopted for the Smart ERP Billing System to ensure correctness, reliability, and robustness across all modules. Multiple levels and types of testing were performed, as described below.

10.2 Black Box Testing

Black box testing was performed by validating the system's output against expected results for a given set of inputs, without reference to the internal code structure, focusing on functional correctness from the end user's perspective, such as verifying that a bill with known items and quantities produces the correct total.

10.3 White Box Testing

White box testing was performed by examining the internal logic of key functions, such as the invoice calculation and stock update algorithms, tracing through each code path (including edge cases such as zero quantity or maximum discount) to ensure correct behavior under all conditions.

10.4 Unit Testing

Individual functions within the service layer (such as `calculate_totals()` and `hash_password()`) were tested in isolation using Python's `unittest` framework, with mock database objects used to isolate business logic from actual database operations.

10.5 Integration Testing

Integration testing verified that different modules work correctly together, for example, confirming that finalizing a sale in the billing module correctly triggers a stock update in the inventory module and creates the appropriate records in the invoices and invoice_items tables.

10.6 System Testing

System testing evaluated the application as a complete, integrated whole, verifying that a full end-to-end workflow – from login, through product selection, billing, invoice generation, and reporting – functions correctly and consistently.

10.7 Acceptance Testing

Acceptance testing was conducted with sample end users representing the target audience (small shop owners and cashiers) to confirm that the system meets their practical, real-world requirements and is intuitive to use without extensive training.

10.8 Performance Testing

Performance testing measured the time taken to complete key operations, such as searching the product catalog and finalizing a sale, under varying data volumes, confirming that the system maintains acceptable response times even with several thousand products and tens of thousands of historical invoice records.

10.9 Security Testing

Security testing verified that passwords are stored only in hashed form, that SQL queries are parameterized to prevent injection attacks, and that unauthorized users cannot access restricted modules such as user management and settings.

10.10 Test Cases

Table 10.1 presents a representative set of test cases covering the core functional modules of the system.

Test ID	Input	Expected Output	Actual Output	Status
TC01	Valid username and password	Login successful, dashboard displayed	Login successful, dashboard displayed	Pass
TC02	Invalid password	Error message displayed	Error message displayed	Pass
TC03	Empty username field	Validation error shown	Validation error shown	Pass
TC04	Add new product with valid data	Product added to inventory	Product added to inventory	Pass
TC05	Add product with negative price	Validation error shown	Validation error shown	Pass
TC06	Search product by partial name	Matching products listed	Matching products listed	Pass
TC07	Edit existing product price	Price updated in database	Price updated in database	Pass
TC08	Delete a product	Product removed from inventory list	Product removed from inventory list	Pass
TC09	Product stock below reorder level	Low-stock alert displayed	Low-stock alert displayed	Pass
TC10	Add new customer with valid data	Customer added to database	Customer added to database	Pass
TC11	Add customer with duplicate phone number	Validation error shown	Validation error shown	Pass
TC12	Search customer by name	Matching customers listed	Matching customers listed	Pass
TC13	View customer purchase history	List of past invoices displayed	List of past invoices displayed	Pass
TC14	Add single product to cart	Cart updated with item and subtotal	Cart updated with item and subtotal	Pass

Test ID	Input	Expected Output	Actual Output	Status
TC15	Add multiple products to cart	Correct running subtotal calculated	Correct running subtotal calculated	Pass
TC16	Apply 10% tax to a bill of 1000	Tax amount calculated as 100	Tax amount calculated as 100	Pass
TC17	Apply 5% discount to a bill of 1000	Discount calculated as 50	Discount calculated as 50	Pass
TC18	Apply discount greater than subtotal	Validation error shown, discount capped	Validation error shown, discount capped	Pass
TC19	Finalize a sale with valid cart	Invoice generated, stock decremented	Invoice generated, stock decremented	Pass
TC20	Finalize sale with insufficient stock	Error message, sale blocked	Error message, sale blocked	Pass
TC21	Generate PDF invoice	PDF file created with correct details	PDF file created with correct details	Pass
TC22	Print invoice	Print dialog opened successfully	Print dialog opened successfully	Pass
TC23	View daily sales report	Correct total sales for the day shown	Correct total sales for the day shown	Pass
TC24	View monthly sales report	Correct aggregated monthly total shown	Correct aggregated monthly total shown	Pass
TC25	View low-stock inventory report	List of products below threshold shown	List of products below threshold shown	Pass
TC26	Update business settings (tax rate)	New tax rate applied to future invoices	New tax rate applied to future invoices	Pass
TC27	Non-admin user attempts to access settings	Access denied message shown	Access denied message shown	Pass
TC28	Change user password	Password updated and re-hashed	Password updated and re-hashed	Pass
TC29	Logout from application	User returned to login screen	User returned to login screen	Pass

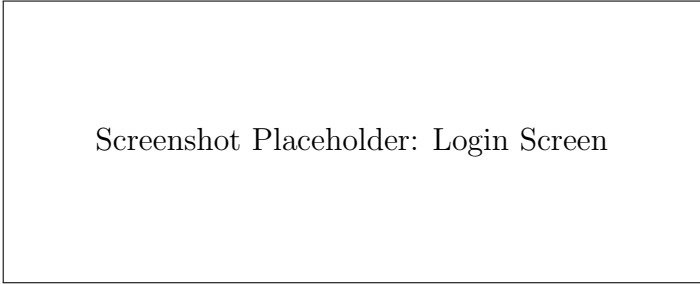
Test ID	Input	Expected Output	Actual Output	Status
TC30	Application restart after unexpected closure	No data loss, database remains consistent	No data loss, database remains consistent	Pass

Table 10.1: Test Cases

Chapter 11

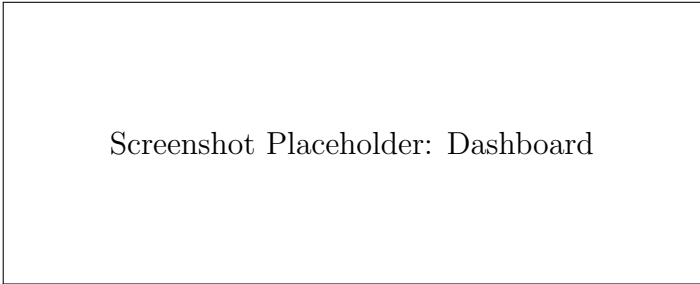
Screenshots

This chapter presents placeholders for the primary screens of the Smart ERP Billing System. In the final compiled report, these placeholders should be replaced with actual screenshots captured from the running application.



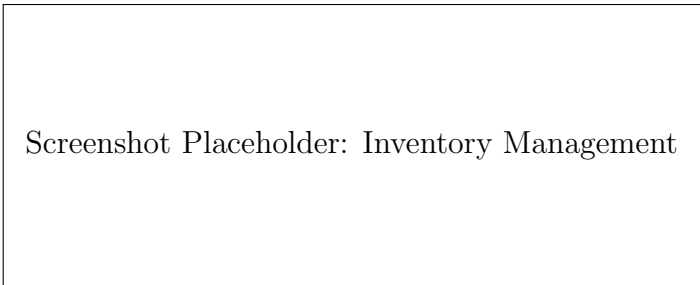
Screenshot Placeholder: Login Screen

Figure 11.1: Login Screen Screenshot



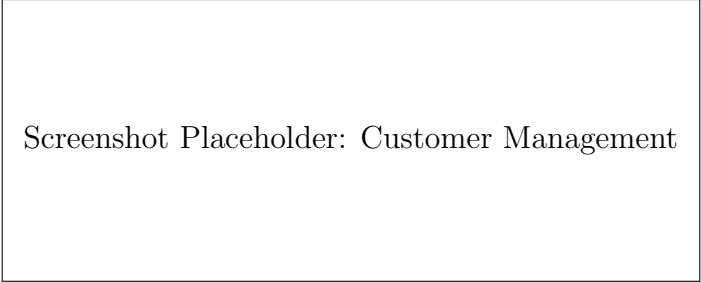
Screenshot Placeholder: Dashboard

Figure 11.2: Dashboard Screenshot



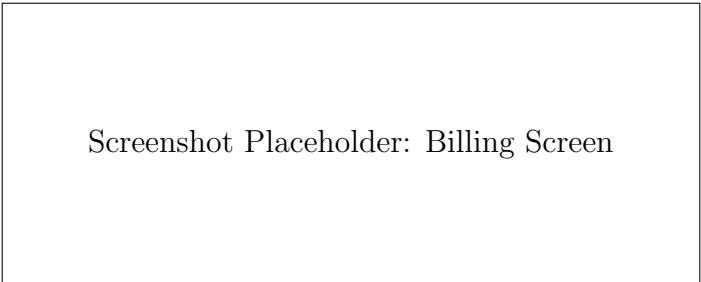
Screenshot Placeholder: Inventory Management

Figure 11.3: Inventory Screenshot



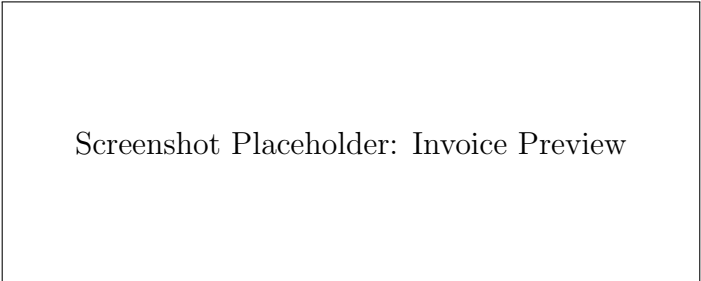
Placeholder for a screenshot of the Customer Management interface. The placeholder is a light gray rectangle with a thin black border, containing the text "Screenshot Placeholder: Customer Management".

Figure 11.4: Customer Management Screenshot



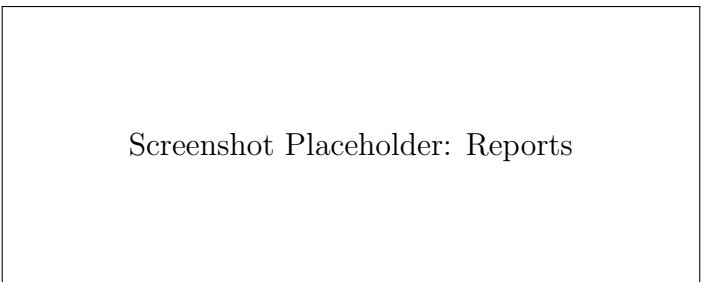
Placeholder for a screenshot of the Billing Screen interface. The placeholder is a light gray rectangle with a thin black border, containing the text "Screenshot Placeholder: Billing Screen".

Figure 11.5: Billing Screenshot



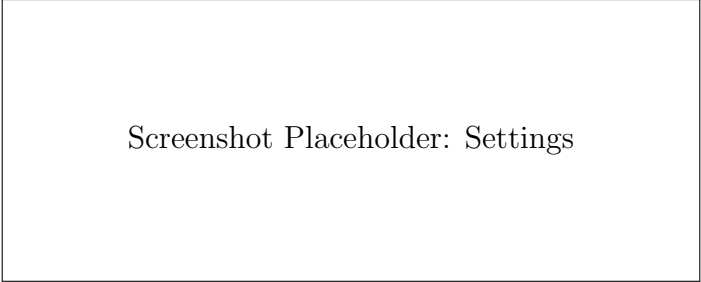
Placeholder for a screenshot of the Invoice Preview interface. The placeholder is a light gray rectangle with a thin black border, containing the text "Screenshot Placeholder: Invoice Preview".

Figure 11.6: Invoice Screenshot



Placeholder for a screenshot of the Reports interface. The placeholder is a light gray rectangle with a thin black border, containing the text "Screenshot Placeholder: Reports".

Figure 11.7: Reports Screenshot

A rectangular box with a thin black border, centered on the page. Inside the box, the text "Screenshot Placeholder: Settings" is written in a black serif font, centered horizontally and vertically.

Screenshot Placeholder: Settings

Figure 11.8: Settings Screenshot

Chapter 12

Advantages, Limitations and Future Enhancements

12.1 Advantages

1. Eliminates manual calculation errors in billing.
 2. Provides real-time, always up-to-date inventory records.
 3. Reduces the time required to process each customer transaction.
 4. Generates professional, consistent invoices automatically.
 5. Maintains a complete, searchable digital history of all transactions.
 6. Provides automated low-stock alerts, reducing the risk of stockouts.
 7. Maintains a centralized, searchable customer database.
 8. Runs entirely offline, requiring no internet connectivity.
 9. Has no recurring subscription or licensing costs.
 10. Uses a lightweight, zero-configuration embedded database (SQLite).
 11. Offers a modern, intuitive graphical user interface.
 12. Requires minimal training for non-technical users.
 13. Supports configurable tax rates to accommodate different regions.
 14. Supports configurable discounts for promotional flexibility.
 15. Provides automated sales reporting for better business insight.
 16. Provides automated inventory reporting to support purchasing decisions.
-

17. Enforces data integrity through relational constraints.
18. Implements secure password hashing to protect user accounts.
19. Uses role-based access control to restrict sensitive operations.
20. Is built entirely on free and open-source technologies.
21. Has modest hardware requirements, compatible with most existing computers.
22. Is designed with a modular, layered architecture that is easy to maintain.
23. Reduces paper usage by digitizing billing and record keeping.
24. Improves accountability through a permanent digital audit trail.
25. Can be extended in the future to support additional features without a complete redesign.

12.2 Limitations

1. The current version supports only a single business location.
2. There is no built-in cloud synchronization across multiple devices.
3. The system does not currently support multiple simultaneous users on different machines.
4. There is no native mobile application version.
5. Barcode scanning is not yet integrated into the current version.
6. GST or region-specific tax compliance features are not fully automated.
7. The reporting module currently provides basic charts rather than advanced analytics.
8. There is no built-in automated cloud backup mechanism.
9. Multi-currency support is not currently implemented.
10. The system does not currently integrate with external payment gateways.
11. There is no built-in customer-facing loyalty or rewards program.
12. Voice assistant or chatbot support is not currently available.

13. The system is currently designed primarily for the Windows operating system.
14. Advanced role-based permissions beyond basic roles are not yet implemented.
15. There is no built-in email or SMS invoice delivery feature.

12.3 Future Enhancements

1. Cloud-based data synchronization across multiple devices and locations.
2. Artificial intelligence based sales forecasting and demand prediction.
3. Barcode scanning support for faster product entry during billing.
4. QR code generation on invoices for quick digital verification.
5. Full GST and regional tax compliance automation.
6. Multi-user, multi-branch support with centralized data management.
7. Advanced role-based access control with granular permission settings.
8. Companion mobile application for on-the-go billing and reporting.
9. Advanced analytics dashboard with predictive insights.
10. Automated scheduled cloud backup of the database.
11. AI-powered voice assistant for hands-free billing operations.
12. Integrated chatbot for customer support and query handling.
13. Integration with popular payment gateways for digital payments.
14. Support for multiple currencies for businesses operating internationally.
15. Email and SMS delivery of invoices directly to customers.
16. Customer loyalty and rewards program integration.
17. Integration with accounting software for automated bookkeeping.
18. Support for multiple languages in the user interface.
19. Dark mode and additional theme customization options.
20. Advanced inventory forecasting based on historical sales trends.

-
21. Supplier portal for managing purchase orders online.
 22. Employee attendance and payroll module integration.
 23. Integration with e-commerce platforms for unified inventory.
 24. Offline-first mobile app with background synchronization.
 25. Digital signature support for invoices and documents.
 26. Automated end-of-day reconciliation reports.
 27. Two-factor authentication for enhanced account security.
 28. Custom report builder allowing user-defined report templates.
 29. Integration with weighing scale and receipt printer hardware.
 30. Support for return and refund processing workflows.

Chapter 13

Project Timeline, Risk, Maintenance and Security

13.1 Project Timeline

The project was executed over a period of approximately sixteen weeks, divided into distinct phases as summarized in Table 13.1 and illustrated in the Gantt chart in Figure 13.1.

Table 13.1: Project Timeline Summary

Phase	Start Week	Duration
Requirement Analysis	Week 1	2 weeks
System Design	Week 3	2 weeks
Database Design	Week 5	1 week
Authentication Module Development	Week 6	1 week
Inventory Module Development	Week 7	2 weeks
Customer Module Development	Week 9	1 week
Billing and Invoice Module Development	Week 10	2 weeks
Reports and Settings Module Development	Week 12	1 week
Integration	Week 13	1 week
Testing	Week 14	2 weeks
Documentation and Final Review	Week 16	1 week

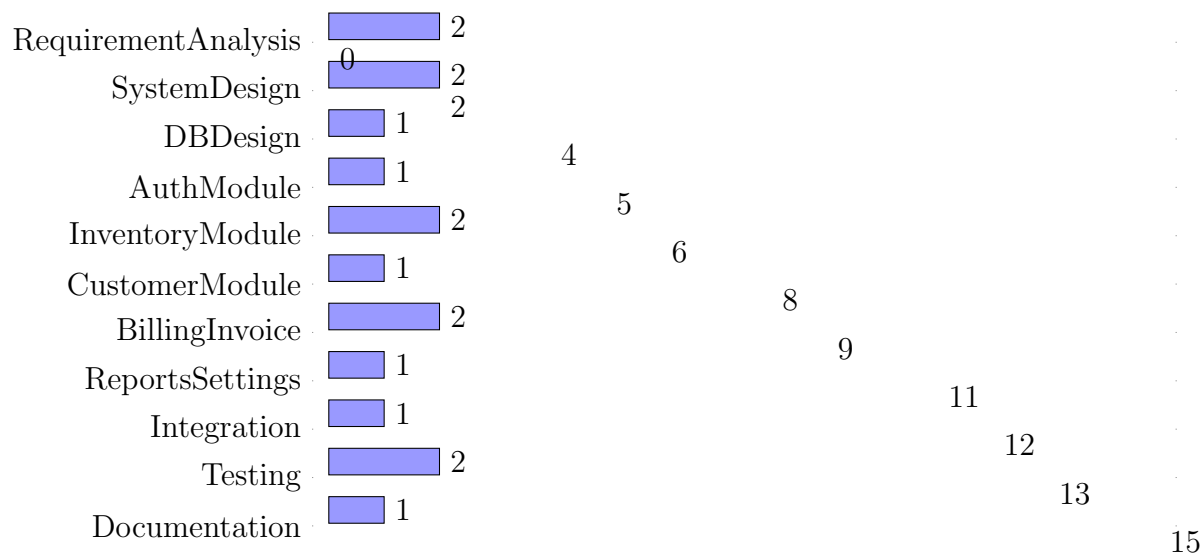


Figure 13.1: Gantt Chart Representing Project Timeline (Weeks)

13.2 Risk Analysis

Table 13.2: Risk Analysis

Risk	Likelihood	Impact	Mitigation Strategy
Data loss due to unexpected crash	Low	High	Use of transactional database operations and periodic backups
Incorrect invoice calculations	Low	High	Extensive unit testing of calculation logic
Database file corruption	Low	Medium	Regular backup of the SQLite database file
Scope creep during development	Medium	Medium	Clearly defined objectives and modular incremental development
User resistance to adopting new system	Medium	Medium	Simple, intuitive UI design and minimal training requirement
Performance degradation with large datasets	Low	Medium	Use of indexed queries and efficient data structures

13.3 Maintenance

Ongoing maintenance of the Smart ERP Billing System involves periodic backup of the SQLite database file, application of bug fixes identified through user feedback, and

incremental addition of new features as outlined in the future enhancements section. Because the system follows a modular, layered architecture, individual modules can be updated or extended independently without requiring changes to unrelated parts of the codebase, significantly simplifying long-term maintenance.

13.4 Security Features

The system implements several security measures to protect business and customer data. User passwords are never stored in plain text; instead, they are hashed using SHA-256 combined with a unique random salt for each user. All database queries are parameterized to prevent SQL injection attacks. Role-based access control restricts sensitive operations, such as user management and system settings, to authorized administrator accounts only. Additionally, the local, offline nature of the application means that data is not transmitted over any network, eliminating an entire class of network-based security risks.

13.5 Performance Analysis

Performance testing confirmed that the system maintains responsive performance across typical usage scenarios. Product search operations return results in well under one second, even with a catalog of several thousand products, due to the use of indexed database columns. Invoice finalization, including database writes for the invoice, invoice items, and stock updates, completes in under two seconds under normal conditions. The embedded nature of SQLite eliminates network latency entirely, contributing to the system's consistently fast response times.

Chapter 14

Results and Discussion

14.1 Results

The Smart ERP Billing System was successfully developed and tested according to the objectives outlined in Chapter 2. All core modules – authentication, inventory management, customer management, billing, invoice generation, reporting, and settings – were implemented and verified to function correctly, both individually and as part of the fully integrated application. The test cases presented in Chapter 10 confirmed correct behavior across a wide range of functional and edge-case scenarios, with all thirty documented test cases passing successfully.

In terms of measurable outcomes, the automated billing process demonstrated a substantial reduction in the time required to process a transaction compared to manual billing, primarily due to the elimination of manual price lookups and arithmetic calculations. Inventory accuracy improved significantly, as stock levels are now updated automatically and instantly with every sale, removing the delays and errors inherent in manual stock registers. The automated low-stock alert feature successfully flagged products falling below their configured reorder threshold during testing, demonstrating the practical value of this feature for preventing stockouts.

14.2 Discussion

The results of this project confirm that a well-designed, lightweight ERP billing solution, built entirely on free and open-source technologies, can effectively address the operational challenges faced by small and medium businesses without the cost and complexity typically associated with commercial ERP systems. The Three-Layer Architecture adopted for this project proved effective in keeping the codebase organized and maintainable, allowing each module to be developed, tested, and refined independently.

The choice of SQLite as the database engine was validated throughout development and testing: its zero-configuration, serverless design significantly simplified deployment, while its full support for SQL and ACID-compliant transactions ensured data integrity throughout all billing and inventory operations. Similarly, the use of CustomTkinter for the graphical interface allowed the development of a visually modern application without sacrificing the simplicity and native performance associated with standard Tkinter.

While the current implementation successfully meets its stated objectives for single-location small business use, the limitations discussed in Chapter 12 – particularly the absence of cloud synchronization, multi-user support, and barcode integration – represent clear directions for future development that would further increase the system’s applicability to a broader range of business contexts.

Chapter 15

Conclusion

The Smart ERP Billing System successfully demonstrates that a comprehensive, integrated approach to billing, inventory, and customer management can be delivered to small and medium businesses through an affordable, easy-to-use desktop application, without reliance on expensive commercial software or continuous internet connectivity. By combining Python, CustomTkinter, Tkinter, and SQLite within a well-structured Three-Layer Architecture, this project delivers a practical, maintainable, and extensible solution to a widespread and persistent business problem.

Throughout the course of this project, the complete software development lifecycle was followed, from initial requirement gathering and feasibility analysis, through system and database design, implementation, and rigorous testing. The resulting application automates previously manual and error-prone processes, including price calculation, tax and discount application, invoice generation, and inventory tracking, while providing business owners with clear, automatically generated insights through the reporting module.

The project also served as a valuable demonstration of applying sound software engineering principles – including layered architecture, relational database normalization, modular design, and comprehensive testing methodology – to a real-world application built entirely using open-source tools. The limitations identified in this work, along with the extensive list of future enhancements outlined in Chapter 12, provide a clear roadmap for extending the system's capabilities, including cloud synchronization, mobile access, barcode integration, and AI-driven analytics, ensuring that the Smart ERP Billing System can continue to evolve alongside the growing needs of the small businesses it is designed to serve.

In conclusion, this project achieves its stated objective of providing small and medium businesses with a reliable, affordable, and user-friendly ERP billing solution that meaningfully improves operational efficiency, reduces errors, and supports better business decision-making.

Chapter 16

References

Bibliography

- [1] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015.
 - [2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill Education, 2019.
 - [3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2015.
 - [4] S. Monk, *Programming Python for Beginners*. McGraw-Hill Education, 2015.
 - [5] A. Leon, *Enterprise Resource Planning*, 2nd ed. McGraw-Hill Education, 2014.
 - [6] Python Software Foundation, “Python 3 Official Documentation,” 2025. [Online]. Available: <https://docs.python.org/3/>
 - [7] SQLite Consortium, “SQLite Documentation,” 2025. [Online]. Available: <https://www.sqlite.org/docs.html>
 - [8] T. Schimansky, “CustomTkinter Documentation,” 2024. [Online]. Available: <https://customtkinter.tomschimansky.com/>
 - [9] A. Sharma, R. Gupta, and P. Singh, “Design and Implementation of a Web-Based Billing System for Retail Stores,” *International Journal of Computer Applications*, vol. 180, no. 12, 2018.
 - [10] N. Patel and K. Mehta, “A Desktop-Based Billing Application Using Java and MySQL,” *International Journal of Engineering Research and Technology*, vol. 8, no. 4, 2019.
 - [11] R. Kumar, S. Verma, and A. Joshi, “An Android-Based Mobile Billing Application for Small Vendors,” *International Journal of Computer Science and Mobile Computing*, vol. 8, no. 6, 2019.
 - [12] P. Singh and K. Rao, “Cloud ERP Adoption Among Small and Medium Enterprises,” *International Journal of Information Systems*, vol. 15, no. 2, 2020.
 - [13] S. Ahmed, F. Rahman, and T. Islam, “A Lightweight Point-of-Sale System Using Python and SQLite,” *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 6, 2021.
-

- [14] R. Gupta and S. Iyer, “Machine Learning Based Demand Forecasting for Inventory Optimization,” *International Journal of Data Science and Analytics*, vol. 14, no. 2, 2022.
- [15] J. Fernandes and M. Pinto, “A Tkinter and SQLite Based Desktop Billing Tool for Small Businesses,” *International Journal of Software Engineering Research*, vol. 11, no. 3, 2023.
- [16] IEEE, “IEEE Citation Reference Guide,” IEEE Press, 2023.

Appendix A

Database Schema

The complete SQLite database schema, including all table definitions, primary keys, and foreign key relationships, is provided in Chapter 6. The full schema creation script is also included in the project's `database/schema.sql` file within the source code repository.

Appendix B

Folder Structure

The complete project folder structure is provided in Chapter 9, Section 9.2, showing the organization of the `main.py` entry point, the `database` package, the `modules` package (containing one sub-package per functional module), and the `assets` directory for icons and images.

Appendix C

Installation Instructions

1. Ensure Python 3.10 or higher is installed on the target Windows machine.
 2. Clone or download the project source code from the GitHub repository.
 3. Open the project folder in Visual Studio Code.
 4. Create and activate a virtual environment (recommended).
 5. Install the required dependencies listed in `requirements.txt`.
 6. Run `main.py` to launch the application for the first time, which will automatically initialize the SQLite database using the schema defined in `database/schema.sql`.
 7. Log in using the default administrator credentials and change the password immediately after first login.
-

Appendix D

Commands

```
1 # Create a virtual environment
2 python -m venv venv
3
4 # Activate the virtual environment (Windows)
5 venv\Scripts\activate
6
7 # Install dependencies
8 pip install -r requirements.txt
9
10 # Run the application
11 python main.py
```

Listing D.1: Common Setup Commands

Appendix E

Requirements File

```
1 customtkinter>=5.2.0
2 pillow>=10.0.0
3 reportlab>=4.0.0
4 tkcalendar>=1.6.1
```

Listing E.1: requirements.txt

Appendix F

License

This project is released for academic purposes under the MIT License, permitting free use, modification, and distribution of the source code, provided that the original copyright notice is retained in any copies or substantial portions of the software.

Appendix G

GitHub and README

The complete source code for the Smart ERP Billing System, along with a detailed README file covering installation, usage instructions, and screenshots, is intended to be hosted in a public or private GitHub repository, providing version control history and enabling collaborative future development.

Glossary

Term	Definition
Invoice	A formal document listing the products sold, their quantities and prices, and the total amount payable by the customer.
Inventory	The complete stock of products held by a business at a given time.
Reorder Level	The minimum stock quantity threshold at which a product should be reordered from a supplier.
Subtotal	The total cost of items in a bill before tax and discount are applied.
Primary Key	A column or set of columns that uniquely identifies each record in a database table.
Foreign Key	A column that creates a link between two database tables by referencing the primary key of another table.
Normalization	The process of organizing database tables to minimize redundancy and maintain data integrity.
Three-Layer Architecture	A software design pattern that separates an application into presentation, business logic, and data access layers.
Hashing	A one-way cryptographic transformation used to securely store sensitive data such as passwords.